

计算机组成与结构

(2026春季)

刘 铎

办公室: 信息大楼B822

邮箱: liuduo@cqu.edu.cn

课程钉钉群





当你听到“计算机”，
脑海中浮现的是什么？

■ 笔记本？ ■ 手机？ 🎮 游戏机？ 🗨️ ChatGPT？ 🚗 自动驾驶？

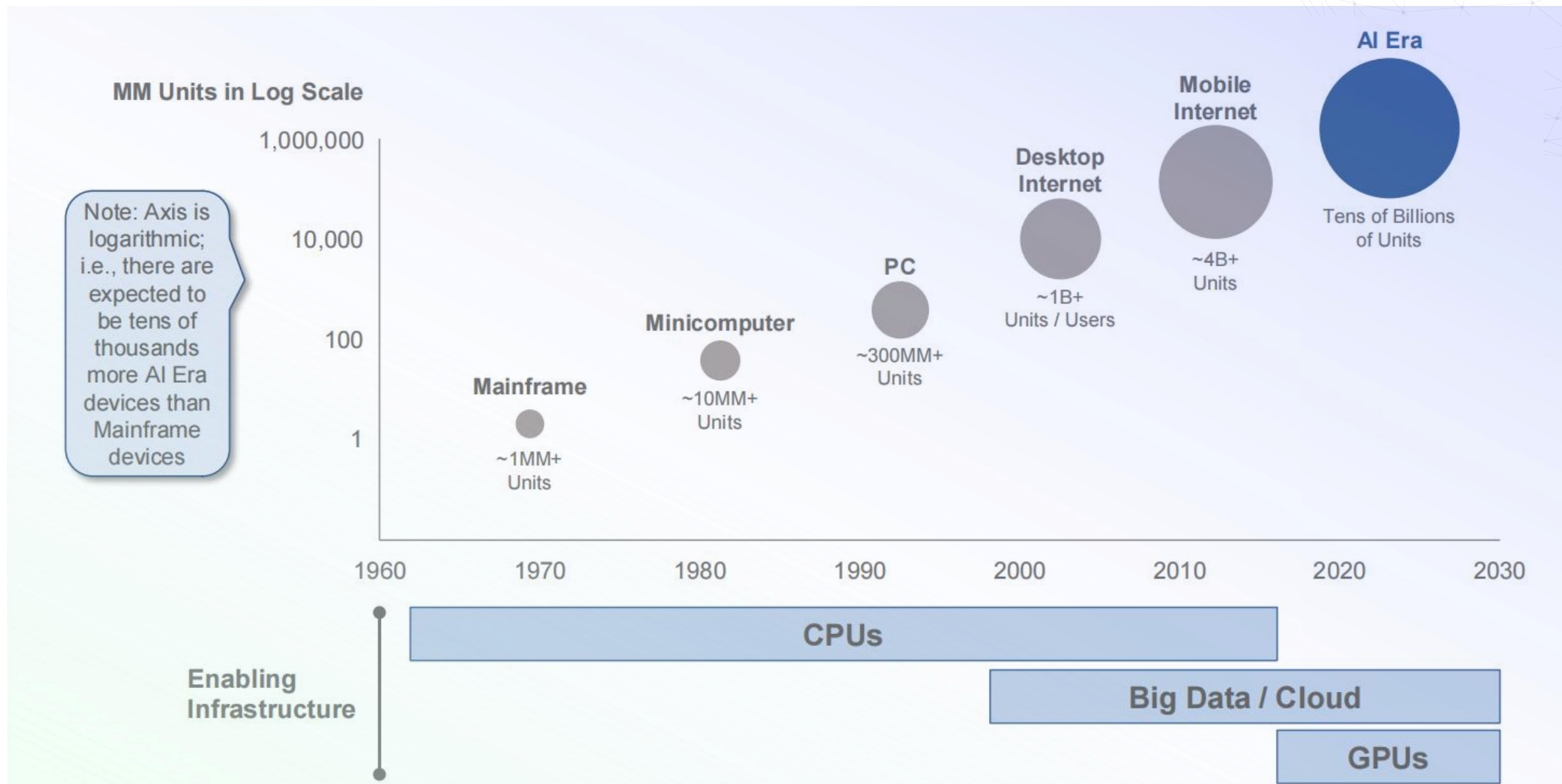


计算机无处不在

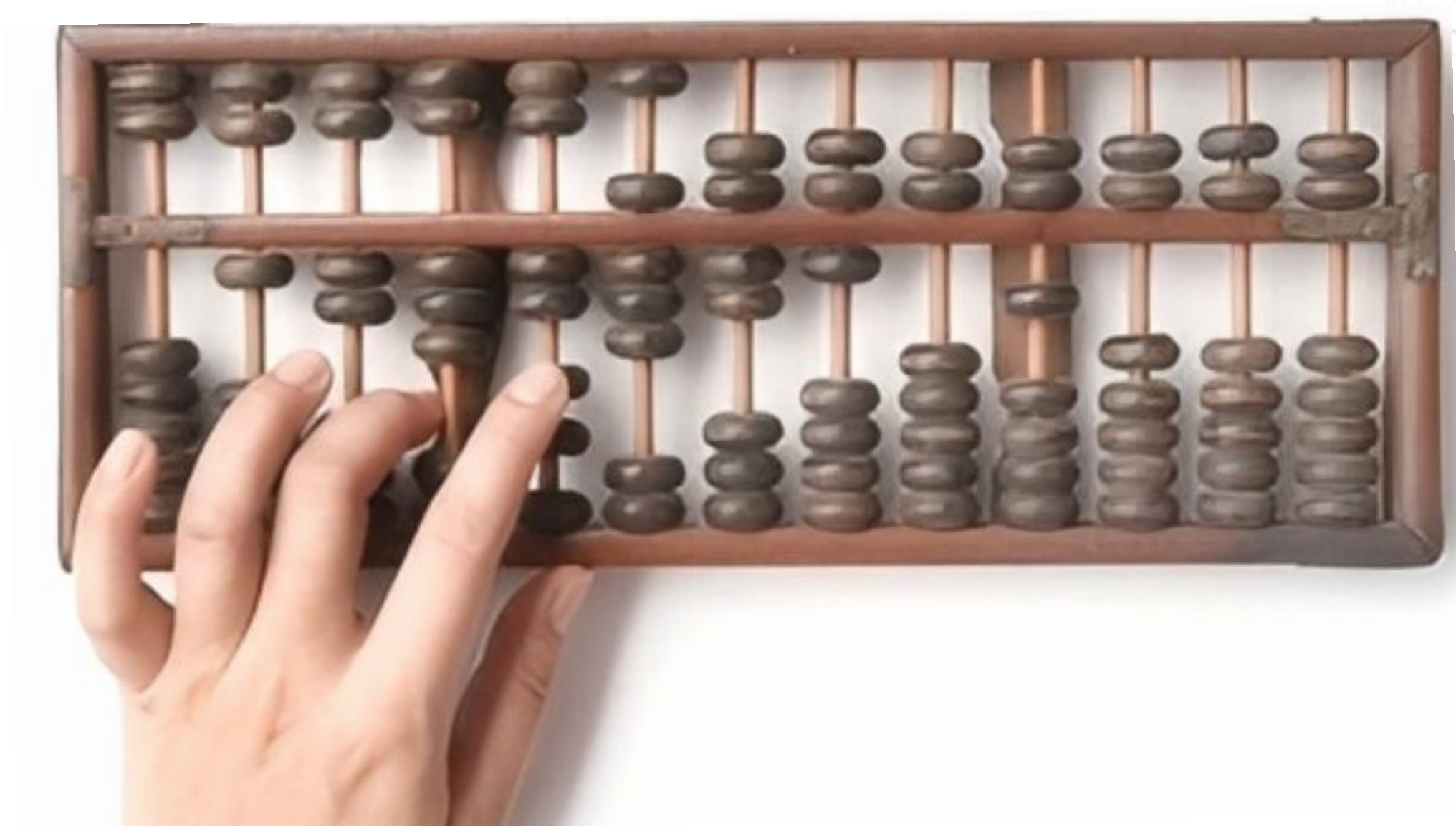
01

计算机发展历程

50年以来计算机的发展



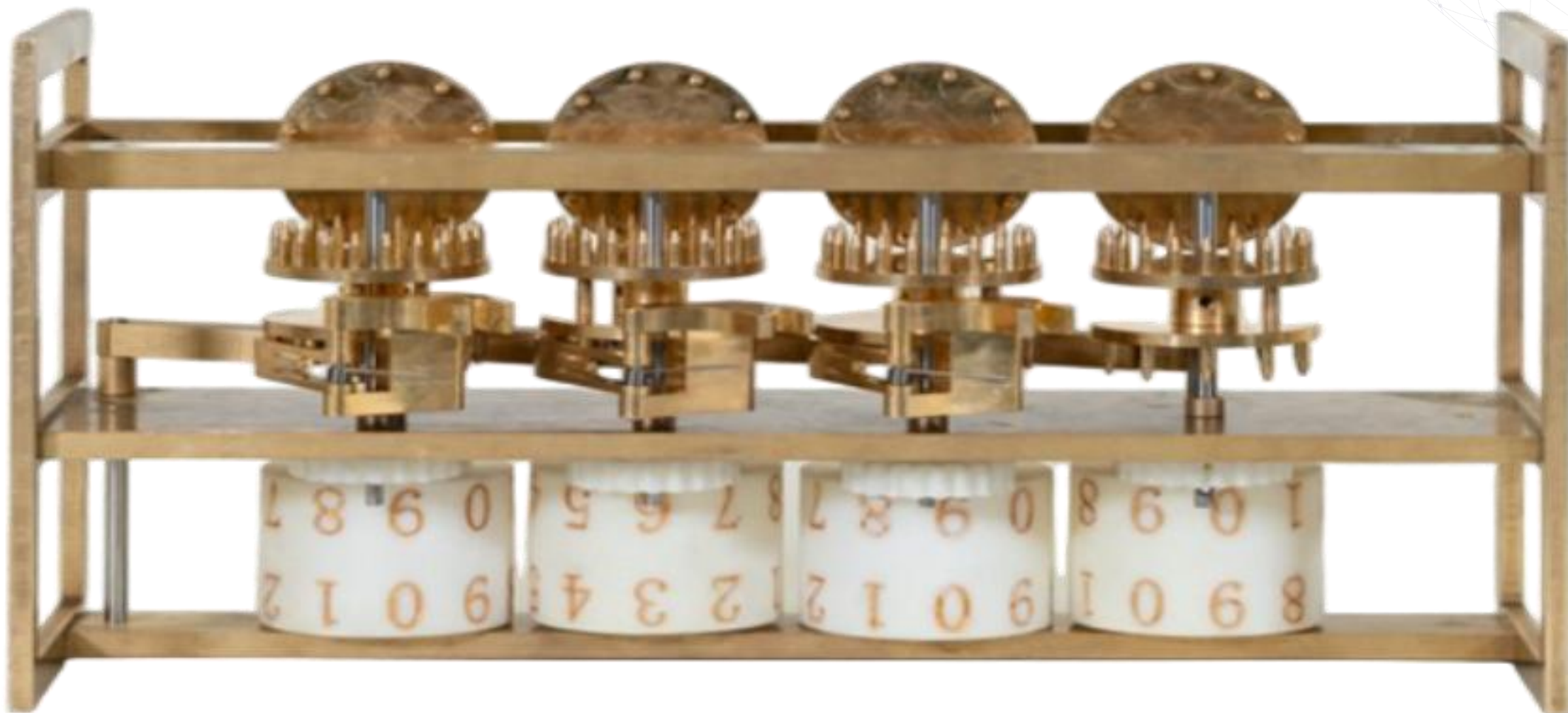
早期计算工具 – 计算机出现之前



面对前苏联专家

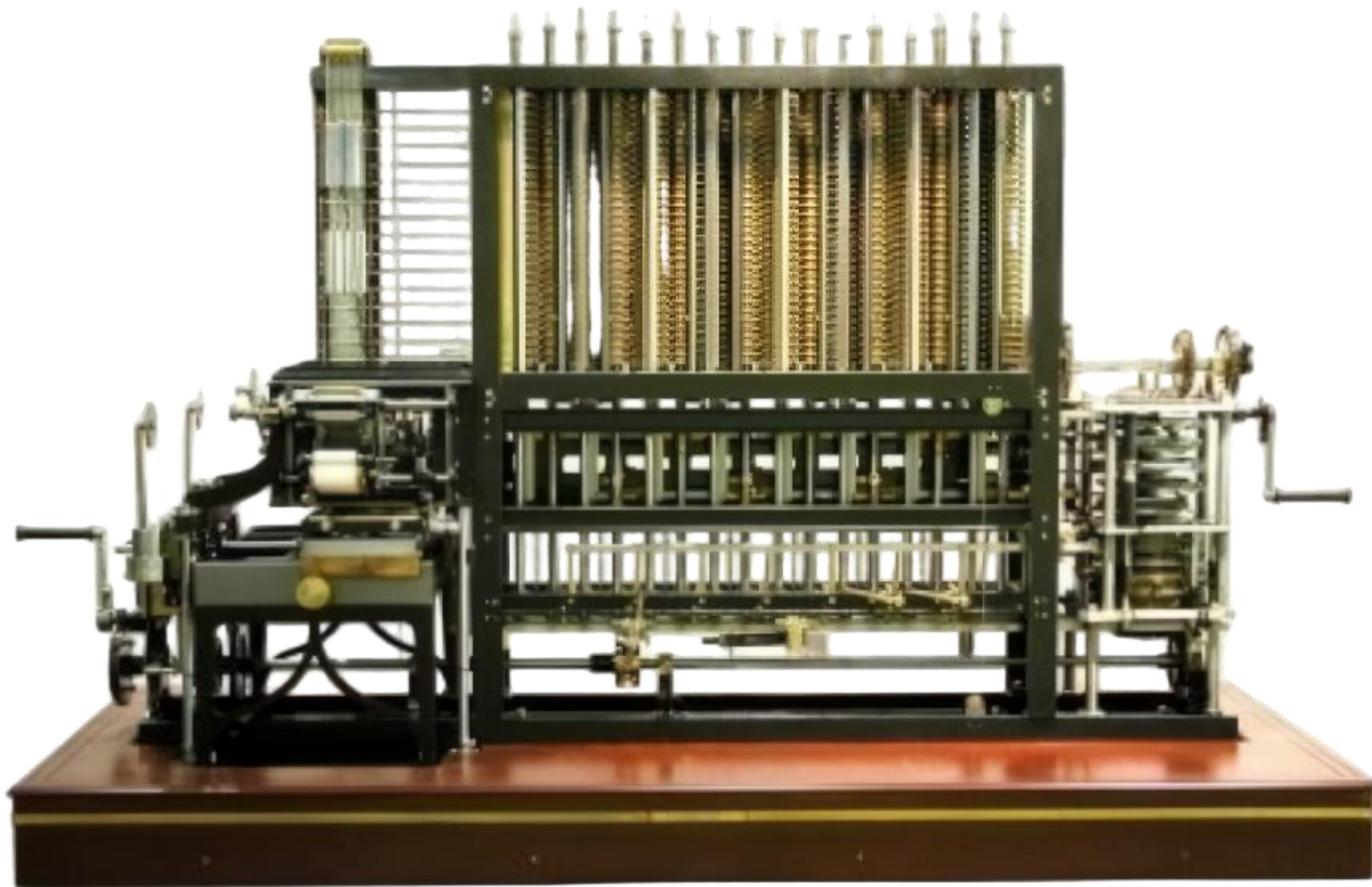
劳动铸就
中国梦

早期计算工具 – 计算机出现之前



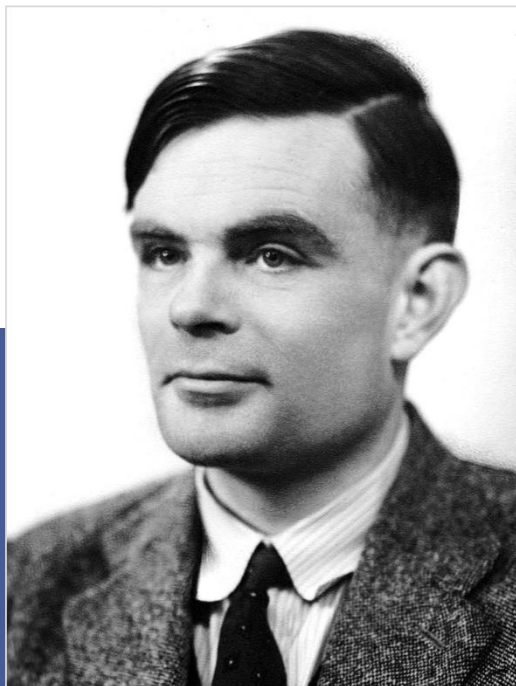
1642年首台机械计算器：帕斯卡加法器

早期计算工具 – 计算机出现之前



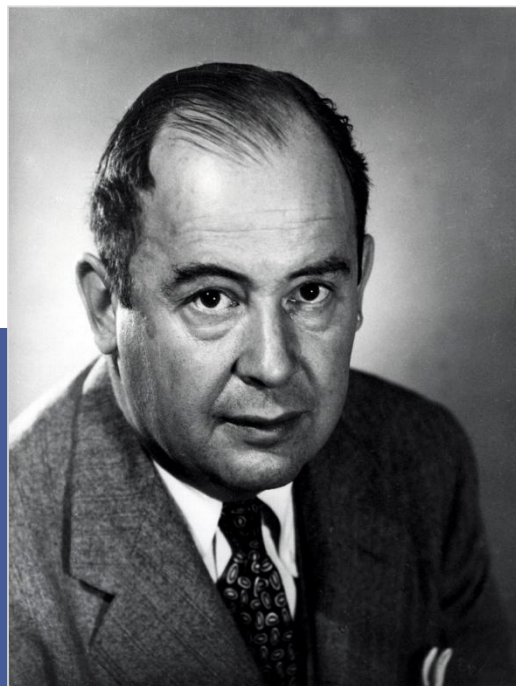
巴贝奇差分机 (1822-1832)

计算机科学奠基人



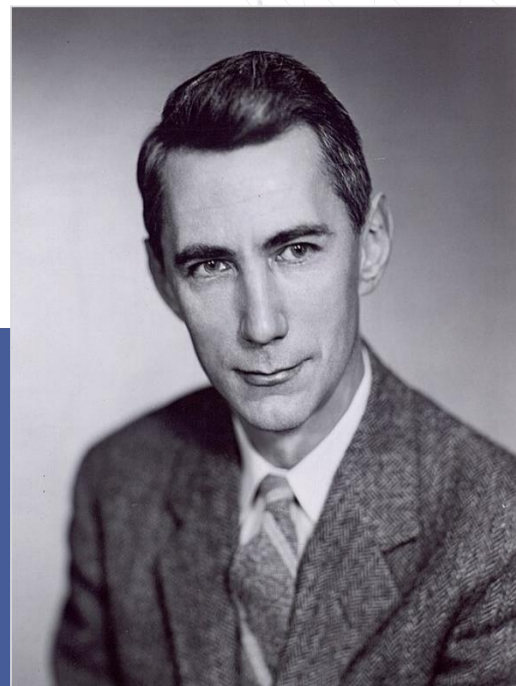
**Alan Turing
(1912-1954)**

**图灵机模型
计算机科学与人工智能之父**



**John von Neumann
(1912-1954)**

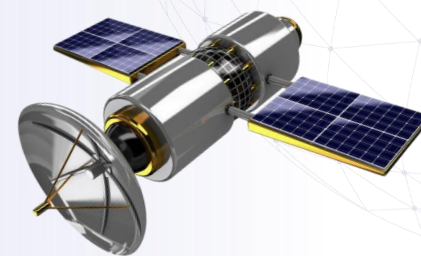
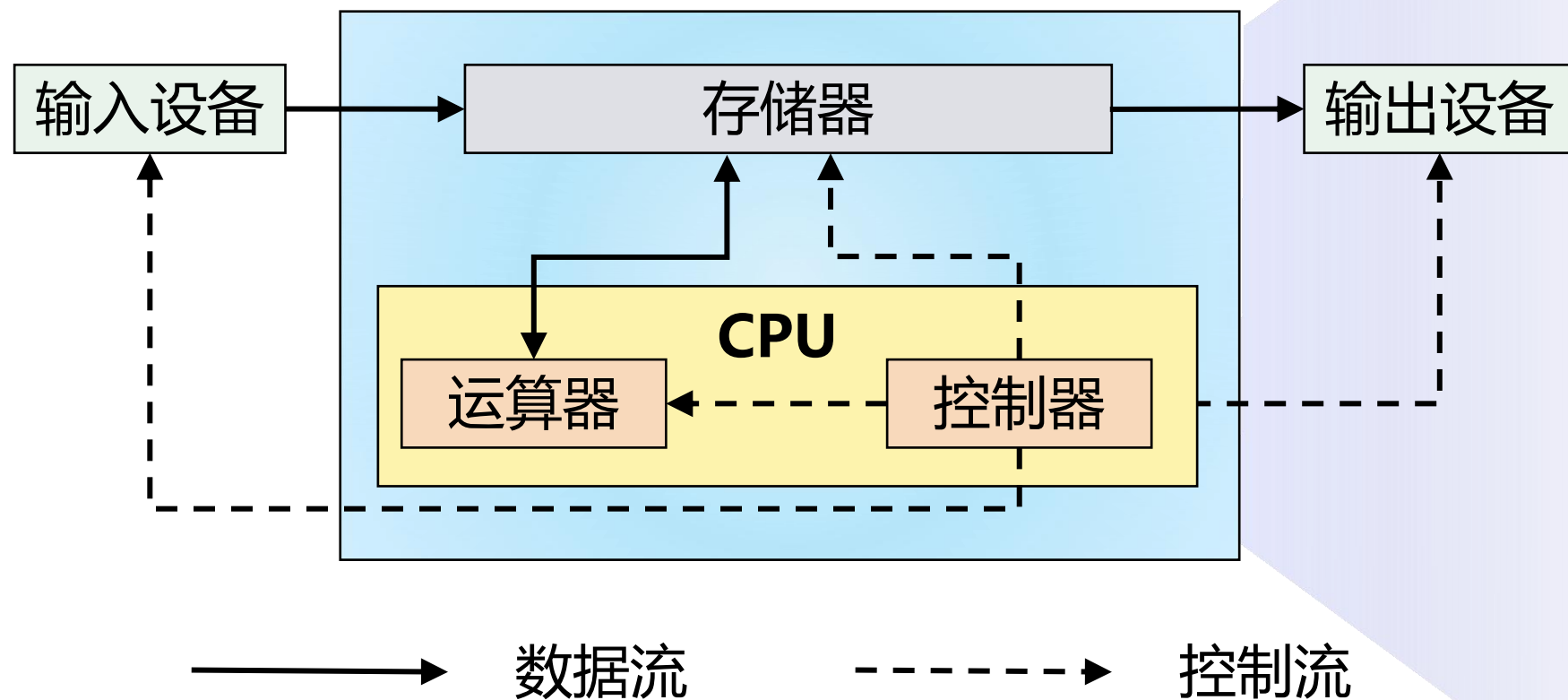
**冯·诺依曼体系架构
计算机科学与博弈论奠基人**



**Claude Elwood Shannon
(1916-2001)**

**通信的数学理论
信息论之父**

冯·诺伊曼体系结构（存储程序计算机）



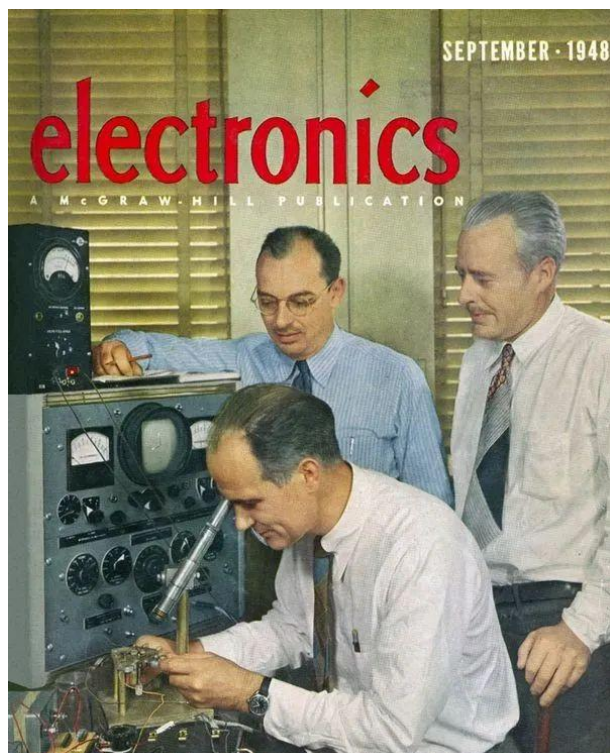
计算机的大脑：集成电路（芯片）

半导体材料
(硅、锗等)

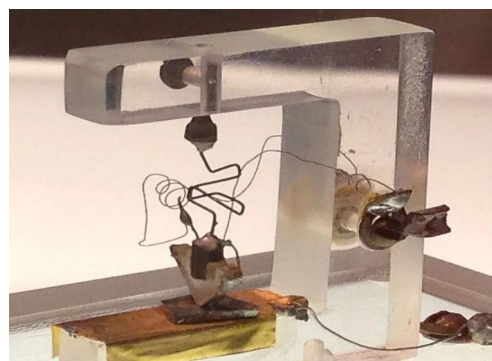
晶体管

集成电路

芯片

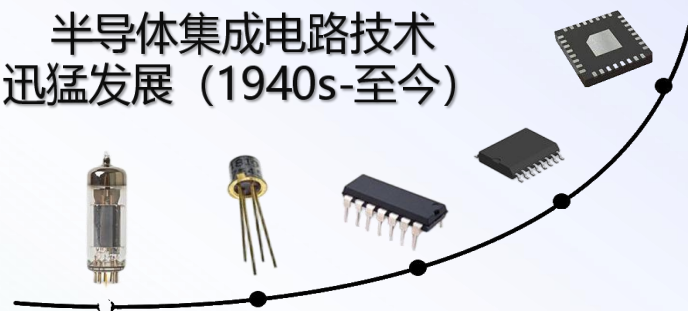


巴丁、布拉顿和肖克利因发明晶体管，获得1956诺贝尔奖



世界上第一个晶体管
(基于锗半导体)

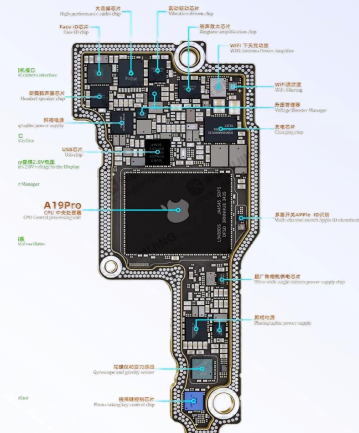
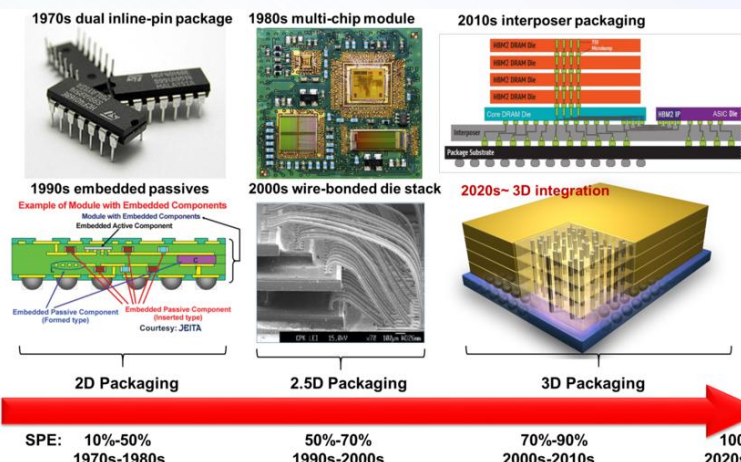
半导体集成电路技术
迅猛发展 (1940s-至今)



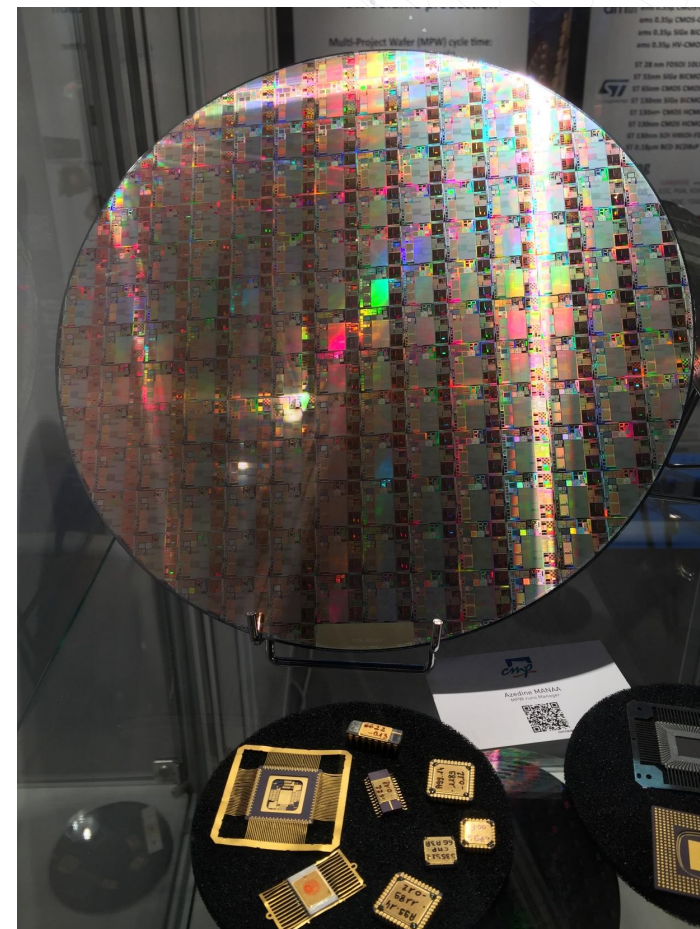
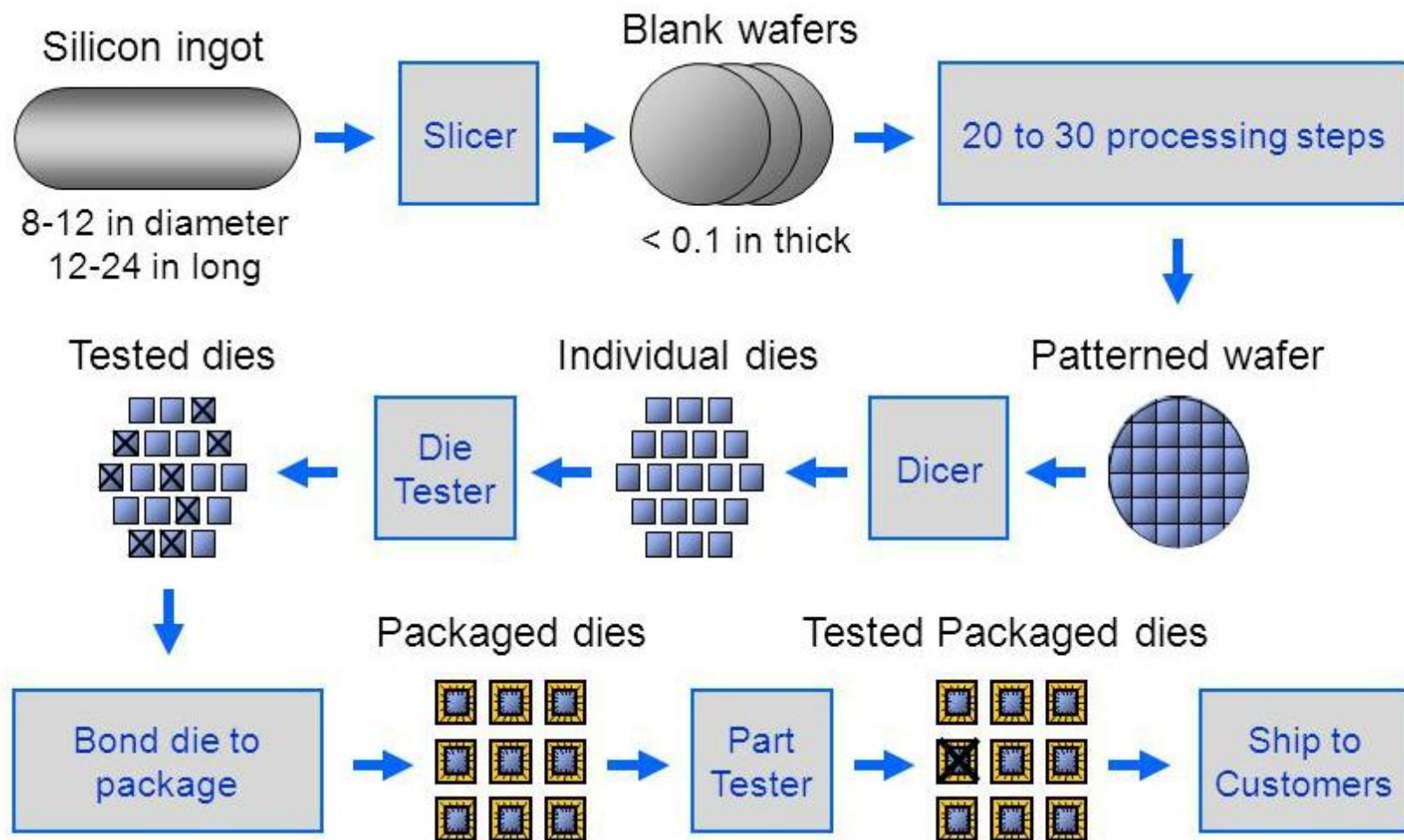
PCB in 1970s



PCB Now



计算机的大脑：集成电路（芯片）制作过程





芯片 制造流程

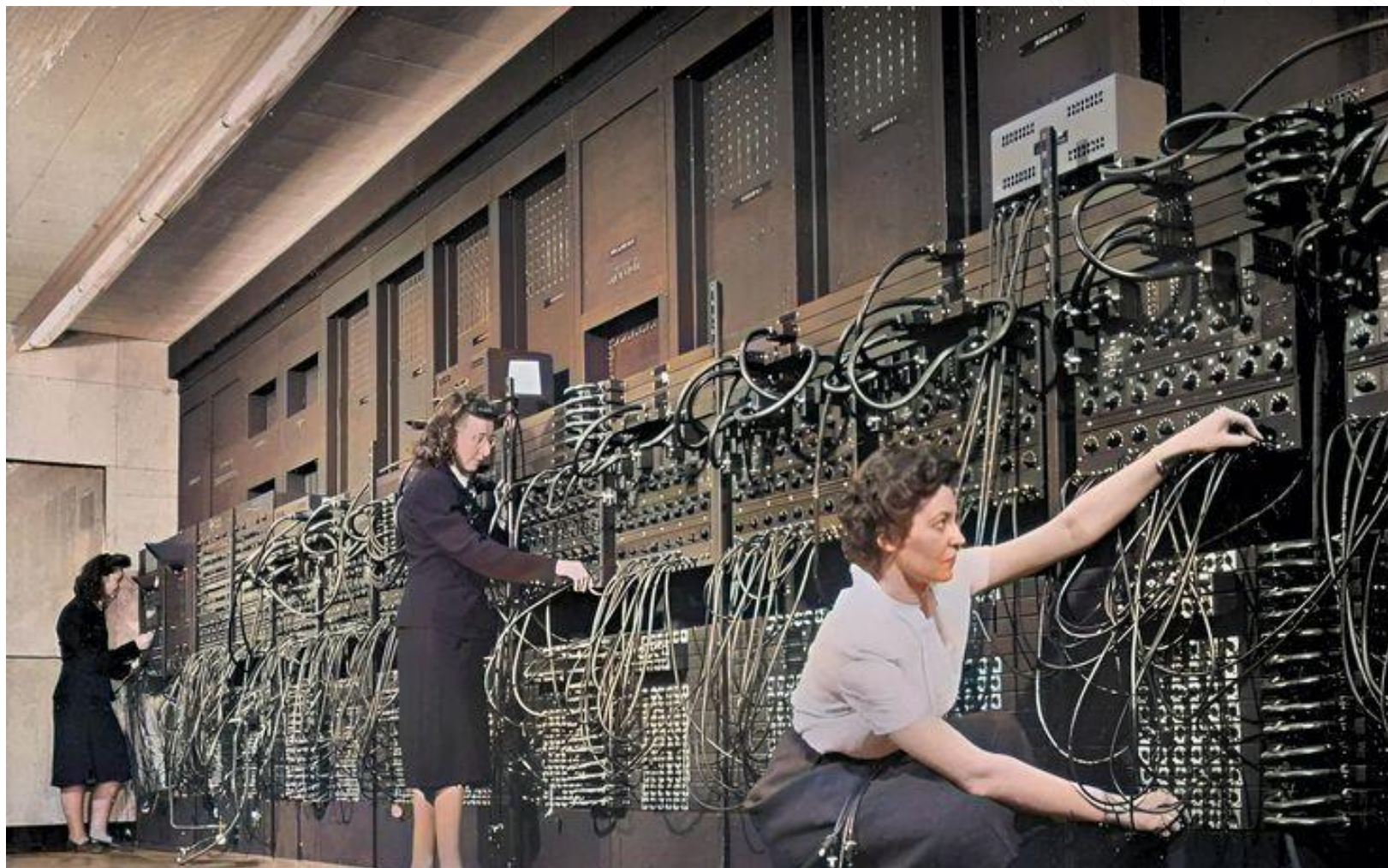
3D动画揭秘芯片 完整制造工艺

计算机发展五代



💡 每一代的飞跃都源于底层器件的革命 — 这就是为什么我们需要理解"组成"

第一代电子计算机 – 计算机的诞生



1945年世界第一台电子计算机ENIAC与女程序员

第一代：电子管时代 (1946-1958)

ENIAC (1946年)

重量：30 吨 (约 5 头大象)

面积：170 平方米 (一个篮球场)

功耗：150 千瓦 (能供 50 户家庭用电)

速度：5000 次加法/秒

18000 个电子管，故障频繁

💡 你的手机计算速度是 ENIAC 的数十亿倍!

30
吨

18,000
个电子管

5,000
次加法/秒

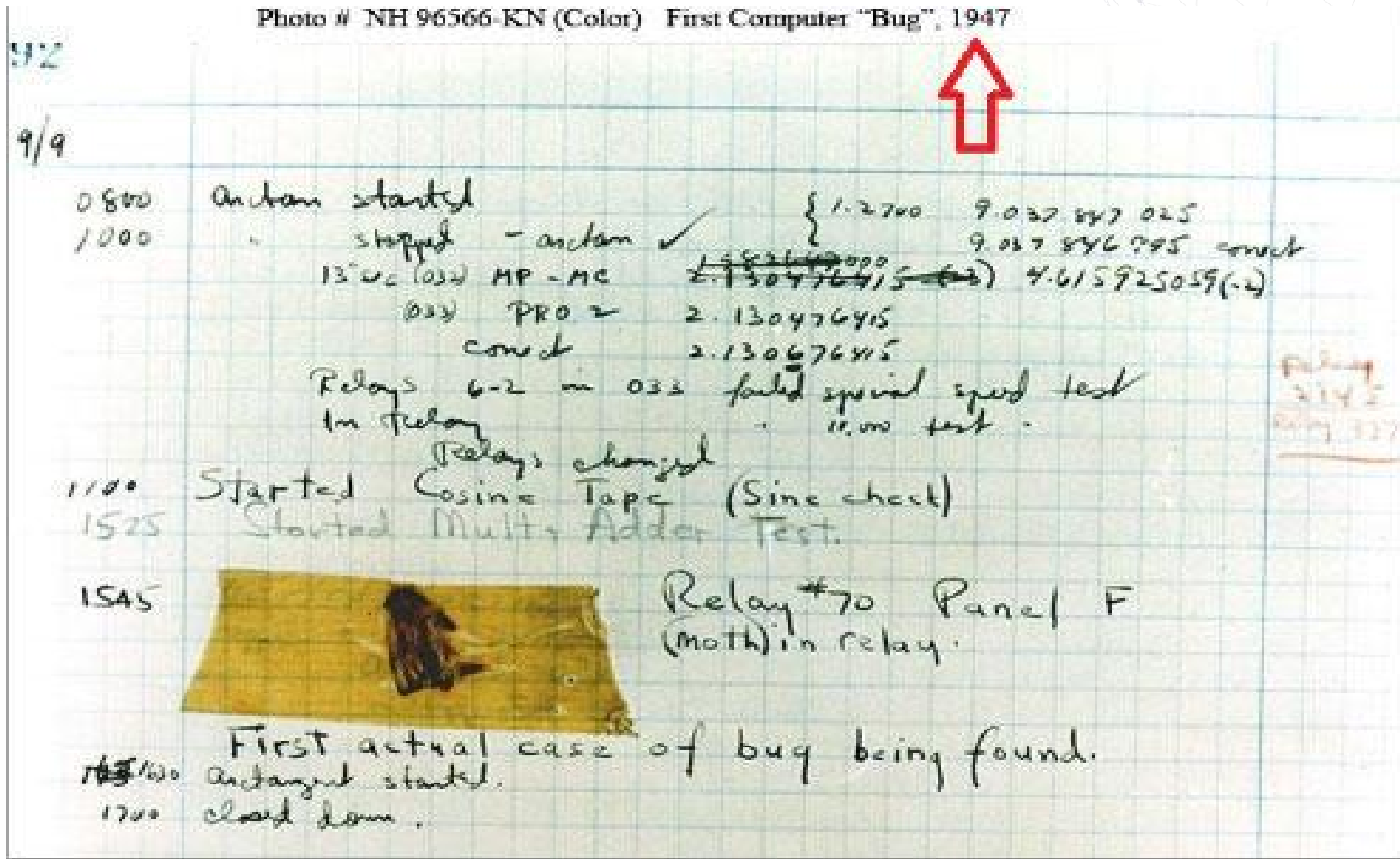
算力的飞跃：从 ENIAC 到 iPhone

	ENIAC (1946)	iPhone 15 Pro (2023)	差距
重量	30 吨	187 克	16万倍
面积	170 m ²	71.5 mm ² 芯片	24亿倍
功耗	150 kW	~5 W	3万倍
运算速度	5000 次/秒	~17万亿次/秒	34亿倍
存储	~200 字节	256 GB	~10亿倍



思考

这些惊人的提升是怎么实现的？答案就在本学期的课程中 — 从晶体管到流水线，从 Cache 到并行计算，每一个知识点都是这个奇迹的一部分。



19

第二至四代：微缩之路



第二代 (1958-1964)

核心技术：晶体管 | 代表：IBM 7090

体积缩小100倍，可靠性大增

衣柜大小



第三代 (1964-1980)

核心技术：集成电路(IC) | 代表：IBM 360

一块芯片集成数百晶体管

办公桌大小



第四代 (1980-至今)

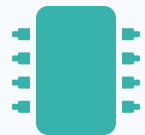
核心技术：超大规模IC | 代表：Intel x86

一块芯片集成数十亿晶体管

指甲盖大小

器件越小 → 速度越快 → 功耗越低 → 成本越低

第五代：AI 与异构计算时代 (2010-至今)



GPU 并行计算

NVIDIA A100/H100 驱动 AI 训练



TPU 专用加速

Google TPU 专为 TensorFlow 设计



NPU 端侧AI

手机/PC 内置 AI 加速单元



RISC-V 开源

开源指令集，打破垄断

关键趋势：从通用处理器 → 领域专用架构 (Domain-Specific Architecture)

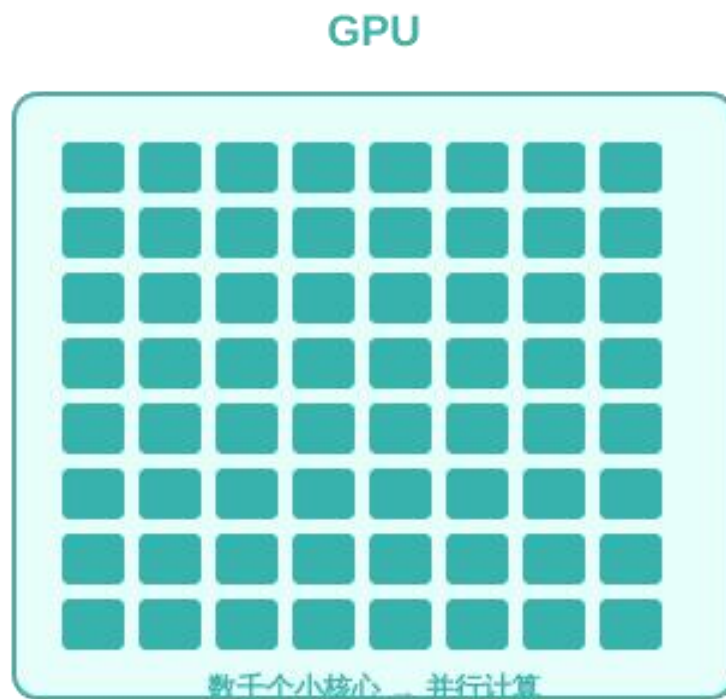
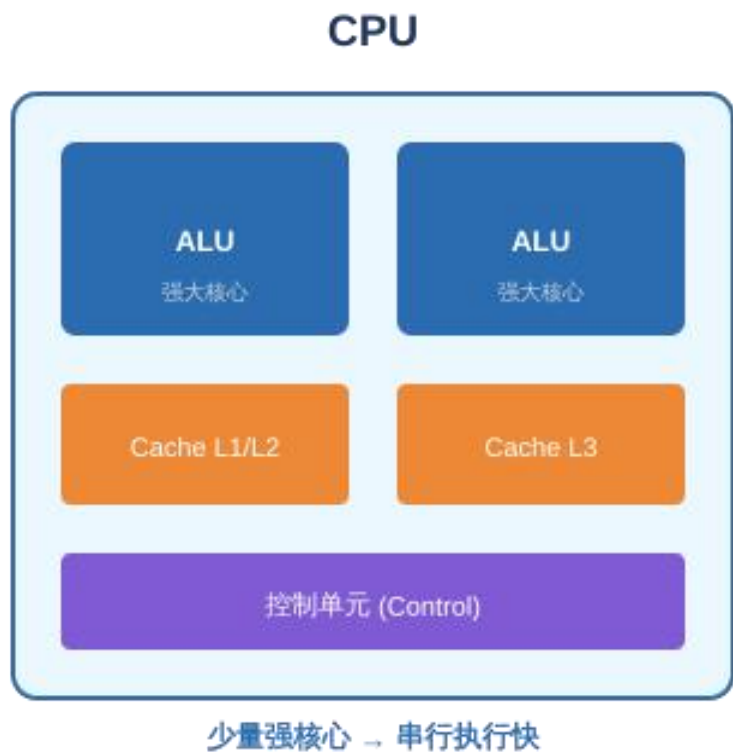
课堂思考

**为什么 AI 训练要用 GPU 而不是 CPU?
GPU 和 CPU 的核心区别是什么?**

思考提示:

1. CPU 擅长什么? GPU 擅长什么?
2. AI 计算的特点是什么? (大量矩阵运算、高并行度)

CPU vs GPU: 为什么 AI 偏爱 GPU?



AI训练 = 海量矩阵乘法

GPU 的**数千个核心**
天然适合并行计算

NVIDIA H100: 16,896 个 CUDA 核心, 训练 GPT-4 使用约 25,000 块, 成本超过 1 亿美元

摩尔定律的诞生

Gordon Moore

Intel 联合创始人

1965年预言：

集成电路上的晶体管数量

大约每两年翻一番

成本几乎不变

这意味着什么？

10年 → 性能提升 ~32倍

20年 → 性能提升 ~1000倍

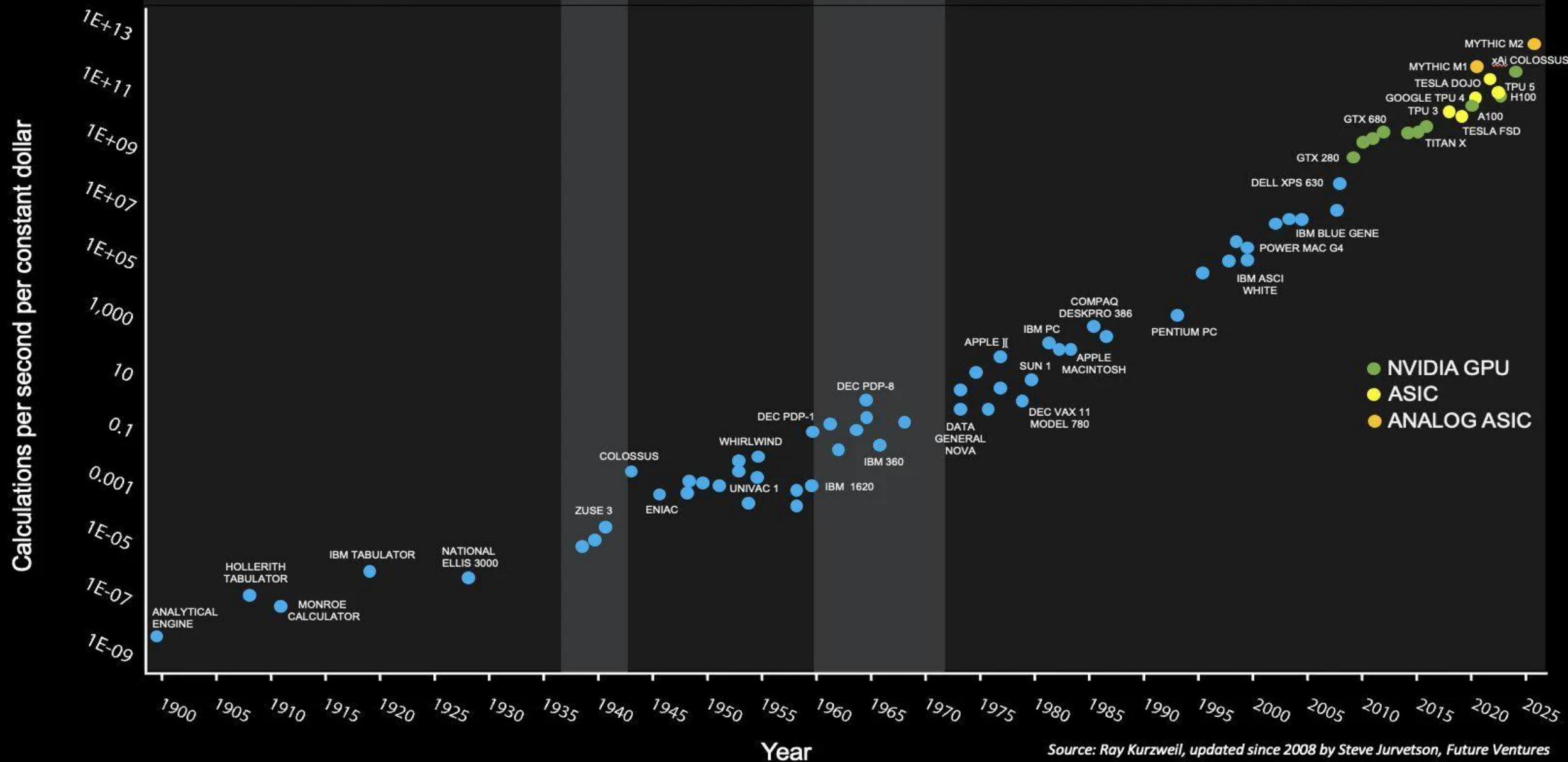
50年 → 性能提升 ~33,000,000倍

类比：

如果汽车按摩尔定律发展，今天的汽车时速将超过光速，而价格只要几分钱！



128 YEARS OF MOORE'S LAW



摩尔定律如何改变了世界

01



个人电脑普及

从百万美元大型机到千元笔记本

02



智能手机革命

手机算力超过90年代超级计算机

03



AI 爆发

算力充足 → 深度学习成为可能

04



云计算

低成本芯片使得数据中心经济可行

没有摩尔定律，就没有今天的数字世界

摩尔定律还有效吗？

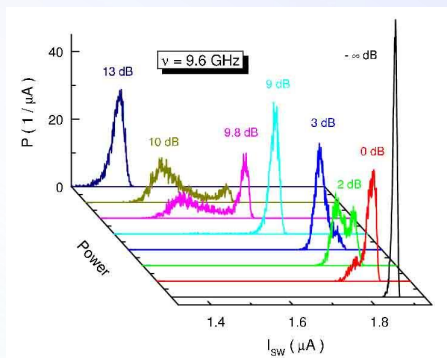
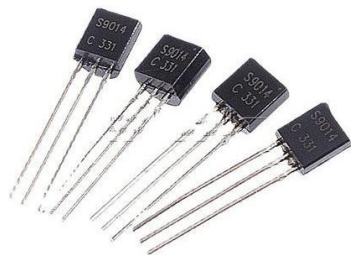
面临的挑战

物理极限：晶体管已接近原子尺度 (3nm~)

功耗墙：密度越高散热越困难

制造成本：先进工艺投资超百亿美元

量子隧穿效应导致漏电



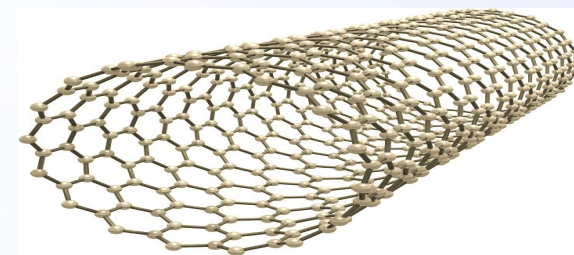
后摩尔时代的出路

领域专用架构 (DSA): GPU、TPU、NPU

Chiplet 技术: 多个小芯片封装一起

3D 堆叠: 纵向增加晶体管数量

新材料/新架构: 碳纳米管、量子计算



摩尔定律的精神 (以更低成本获得更强性能) 仍在延续, 只是路径变了

课堂思考

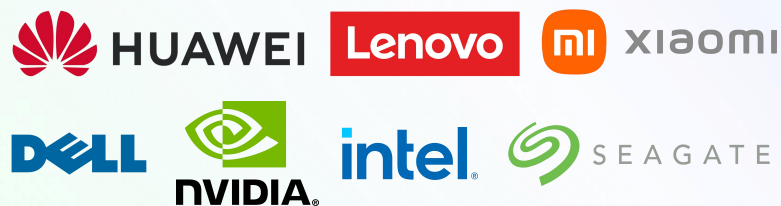
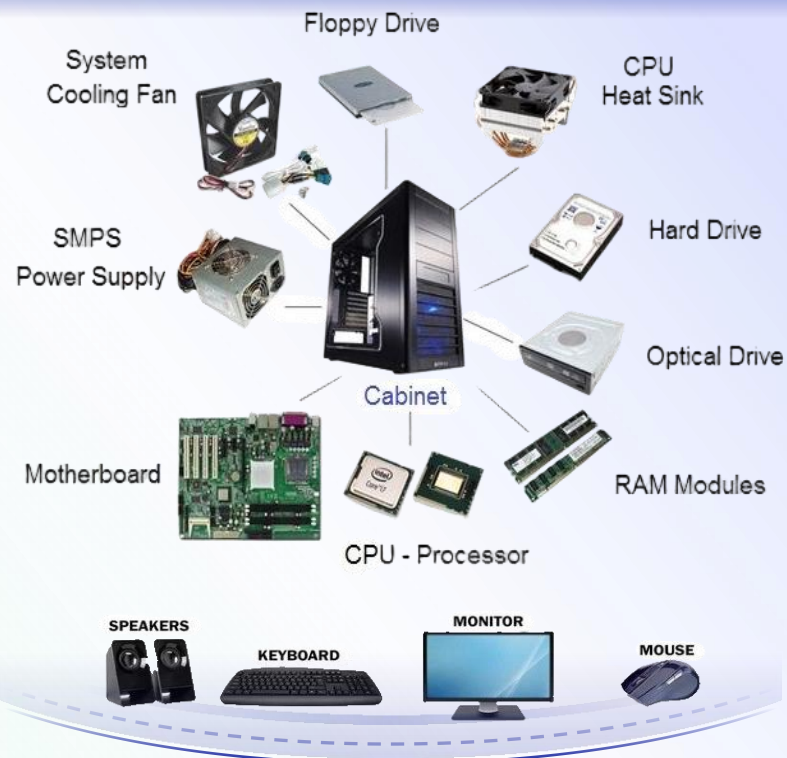
**如果摩尔定律完全失效，会对 AI 发展产生什么影响？
我们还能继续训练越来越大的模型吗？**

思考提示：

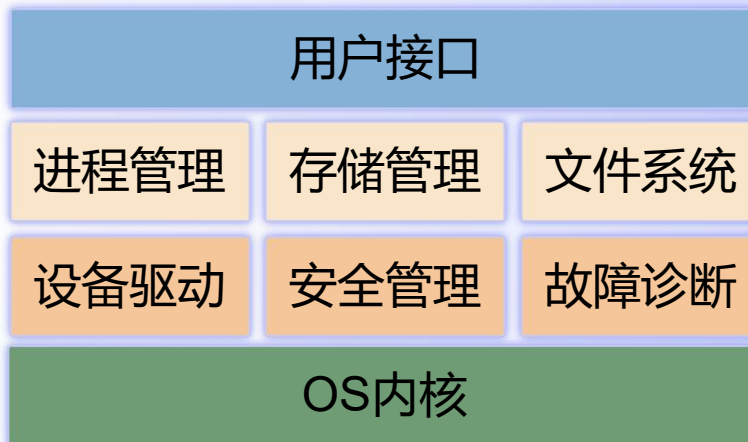
1. 想想 AI 模型参数量增长趋势（GPT-3: 1750亿 → GPT-4: 上万亿）
2. 除了增加晶体管数量，还有什么方法提升 AI 计算效率？
3. 提示：算法优化、稀疏计算、量化技术...

计算机是由什么组成的？

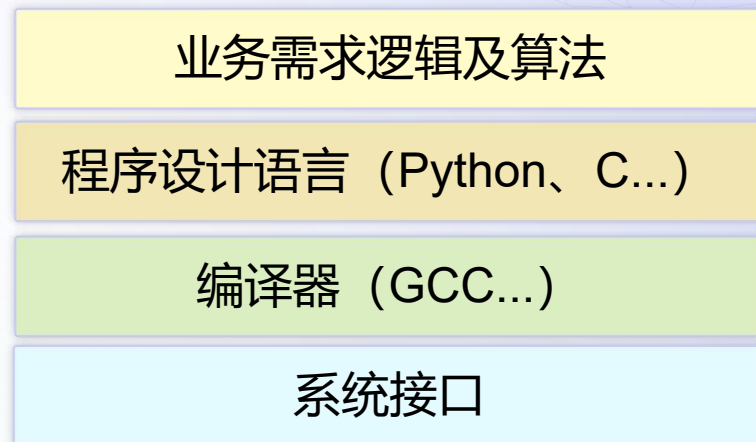
硬件系统



操作系统



应用程序





用戶接口

存儲管理

設備管理

進程管理

用戶界面

文件系統



做完你的我做你的



1970's Computer

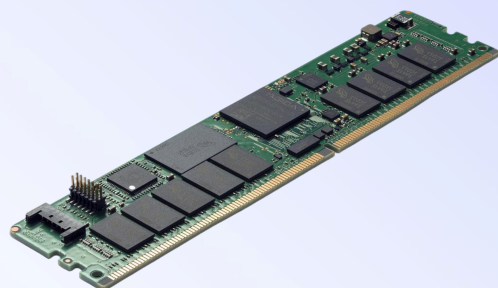
1936 - 2018

关于存储器——现在

内存



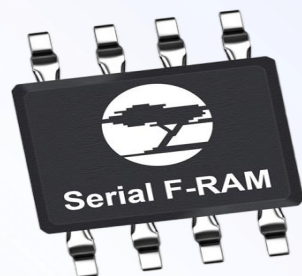
SRAM



DRAM



ROM



FeRAM

外存



机械硬盘



固态硬盘



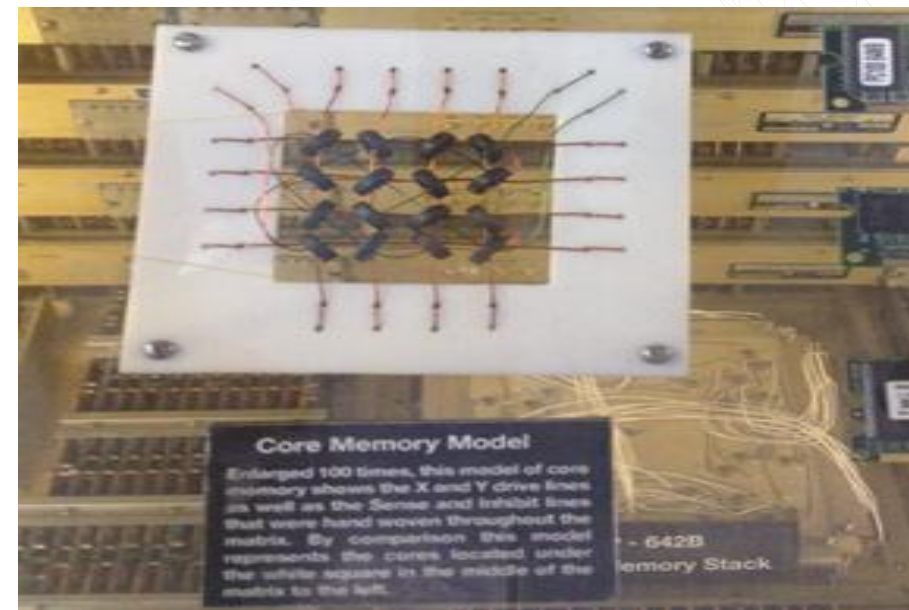
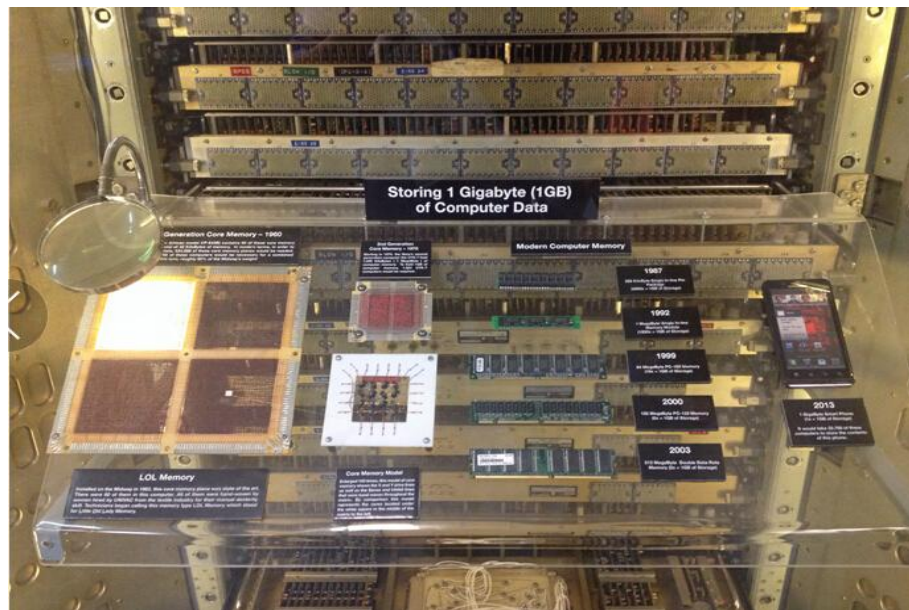
SD卡



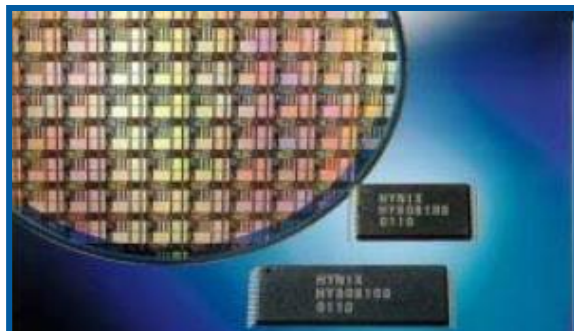
U盘



关于存储器——过去



关于存储器——未来



Ferroelectric RAM (FeRAM)
Toshiba, 2009



Magnetic RAM (MRAM)
EverSpin, 2008



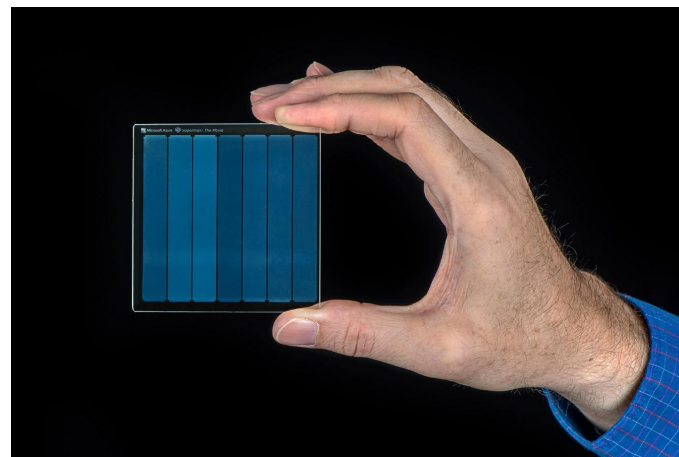
Phase Change Memory (PCM)
Samsung, 2008



Resistive RAM (Memristor)
HP Lab, 2009



DNA存储

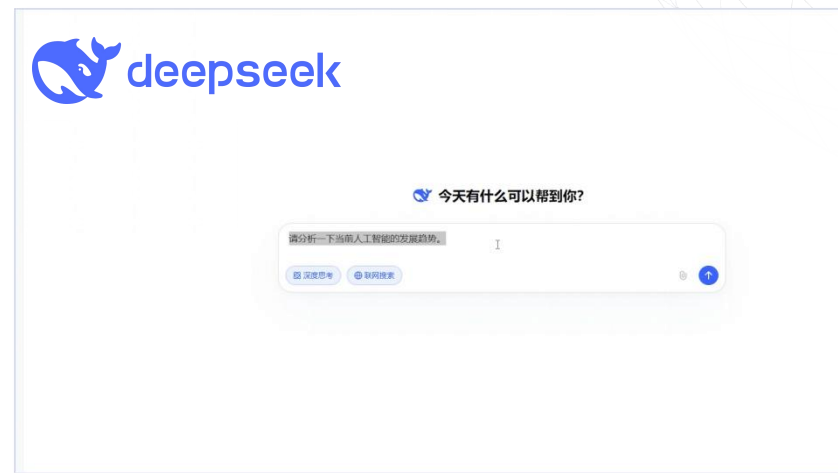


玻璃存储

风起云涌的人工智能热潮



春晚武BOT



DeepSeek AI对话



AI辅助驾驶



AlphaGo对阵李世石



NOBELPRISET I FYSIK 2024 THE NOBEL PRIZE IN PHYSICS 2024



KUNGL.
VETENSKAPS-
AKADEMIEN

THE ROYAL SWEDISH ACADEMY OF SCIENCES

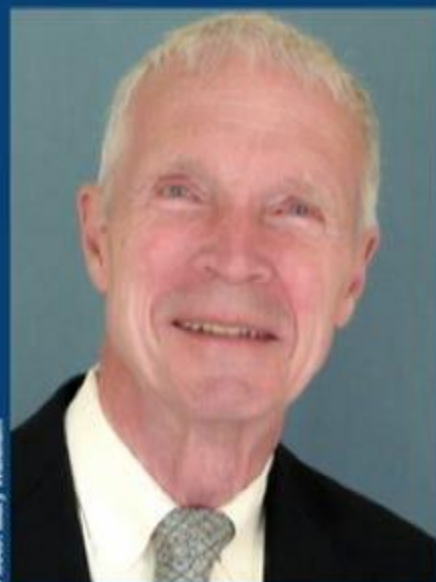


Photo: Mary Wallum

John J. Hopfield

Princeton University, NJ, USA



Photo: University of Toronto

Geoffrey E. Hinton

University of Toronto, Canada

”för grundläggande upptäckter och uppfinningar som möjliggör maskininlärning med artificiella neuronnätverk”

“for foundational discoveries and inventions that enable machine learning with artificial neural networks”

THE
NOBEL
PRIZE



NOBELPRISET I KEMI 2024 THE NOBEL PRIZE IN CHEMISTRY 2024



KUNGL.
VETENSKAPS-
AKADEMIEN

THE ROYAL SWEDISH ACADEMY OF SCIENCES



David Baker
University of Washington
USA

"för datorbaserad proteindesign"

"for computational protein design"



Demis Hassabis
Google DeepMind
United Kingdom

"för proteinstrukturprediktion"

"for protein structure prediction"



John M. Jumper
Google DeepMind
United Kingdom

人工智能第三次浪潮代表性人物



Geoffrey Hinton

坚持模仿大脑而不是
模仿行为

Google AI 前负责人

Hinton的几种算法让
深度神经网络work，
带活了整个领域

“神经网络之父”



Yann LeCun

Facebook AI 前负责人

当今世界AI领域神一
般的人物

“卷积神经网络之父
(CNN)”



Yoshua Bengio

仍坚持科研方向
法国蒙特利尔大学
神经网络复兴的主要
的三个发起人之一

对RNN的一系列推动

算法突破



李飞飞

Standford AI负责人
Google云AI负责人

ImageNet创始人

数据突破

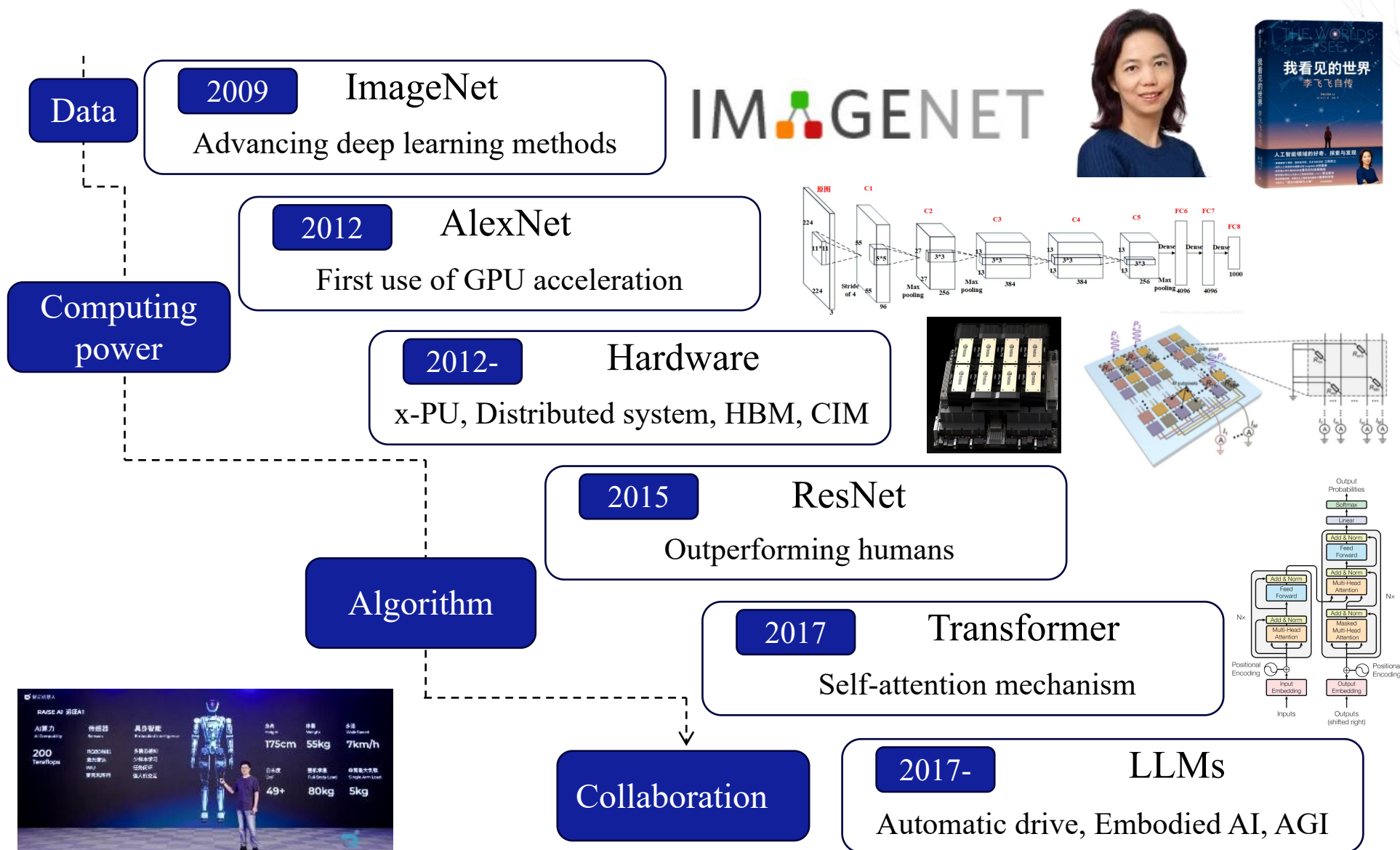


黄仁勋

NVIDIA创始人
GPU市场主导地位

计算突破

从ImageNet到大语言模型



02

课程介绍

2017年图灵奖



Association for Computing Machinery
@TheOfficialACM

...

John L. Hennessy and David A. Patterson, joint recipients of the 2017 [#ACMTuringAward](#), will deliver their [#TuringLecture](#) at @isca2018 on June 4 at 5:00pm Pacific Time.

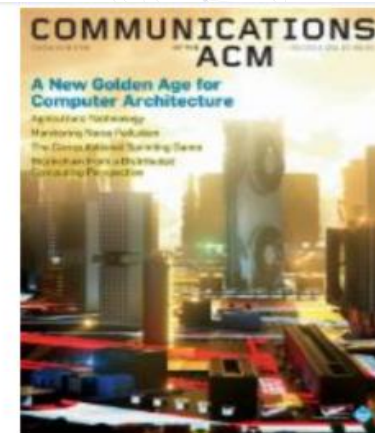
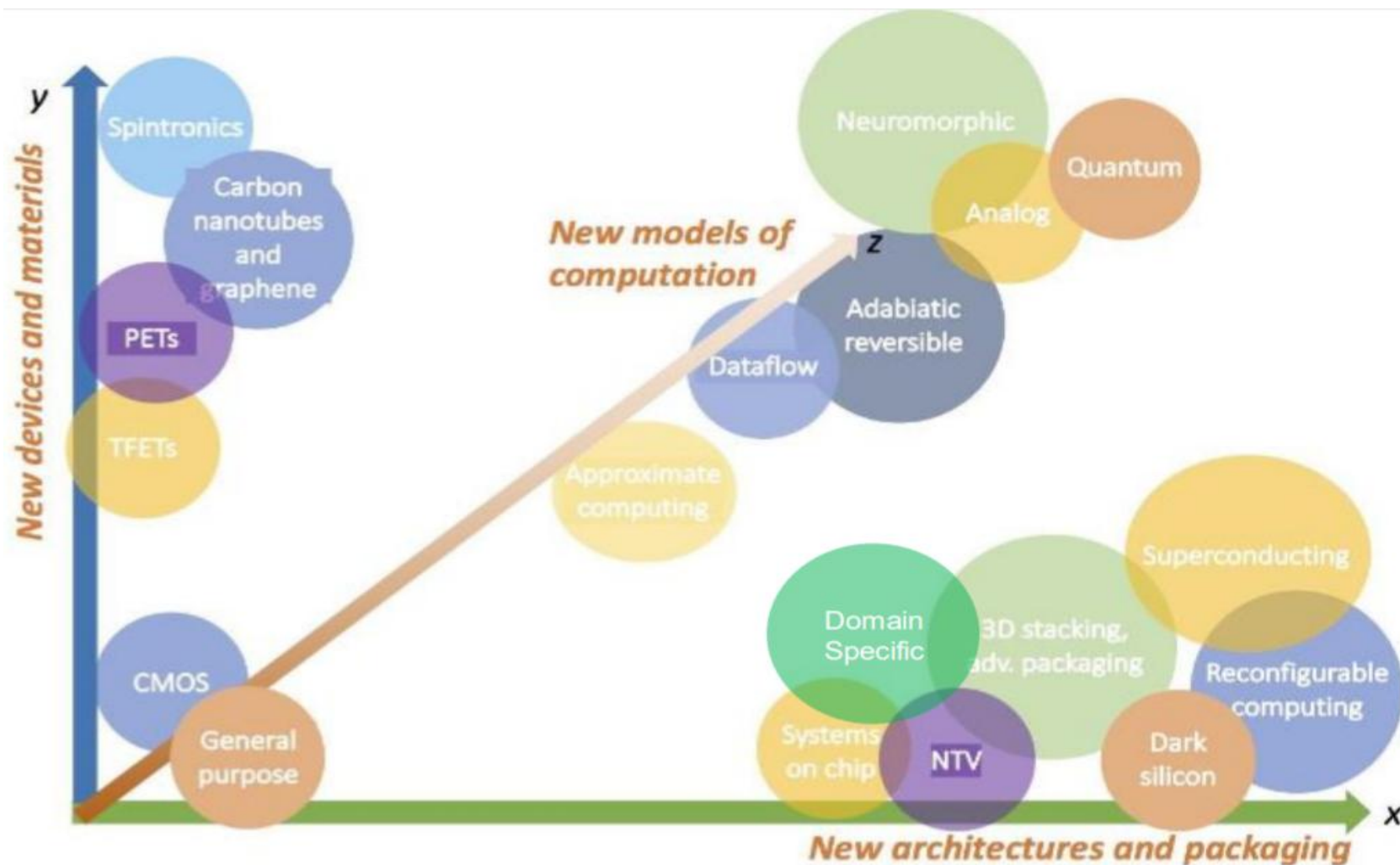
Find out more: ow.ly/61Ph30k3pGK

翻译帖子



计算机体系结构进入新的黄金时代!

计算机体系结构的新黄金时代



AI 时代，为什么还要学计算机组成与结构？

AI的基石

ChatGPT 的每次回答，背后是数万块 GPU 在做矩阵运算



性能关键

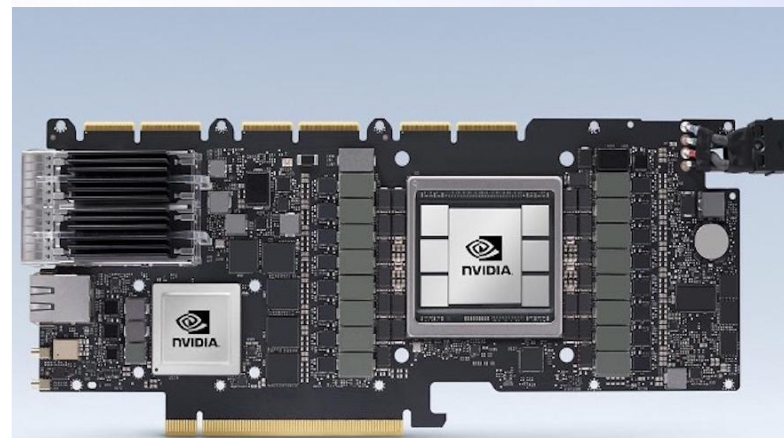
为什么训练大模型需要几周？

	GPU	CPU	
			
	HBM HBM HBM	DRAM	
	NVIDIA A100	Intel Xeon 8280	AMD Rome 7742
Peak FP64 GigaFLOPs	19500	2190	2300
Memory B/W (GB/sec)	1555	131	204
Compute Intensity	100	134	90

瓶颈在内存带宽和计算架构

职业发展

芯片设计、AI 加速器开发是当前最热门的技术方向



理解底层 → 才能驾驭上层

**This is your
AI girlfriend**



**This is your
AI girlfriend
without makeup**



大模型时代的痛点

Llama-3 70B
包含 700 亿
个参数。



痛点：每次生成一个字，都要把 140GB 数据从显存搬进计算核心。

时代进步直观体现——软件的进化



C++/Java 手写逻辑



PyTorch 训练权重



LLM Native & Agents



计算机体系结构的核心逻辑：全面理解性能方程

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{Clock Cycle}$$

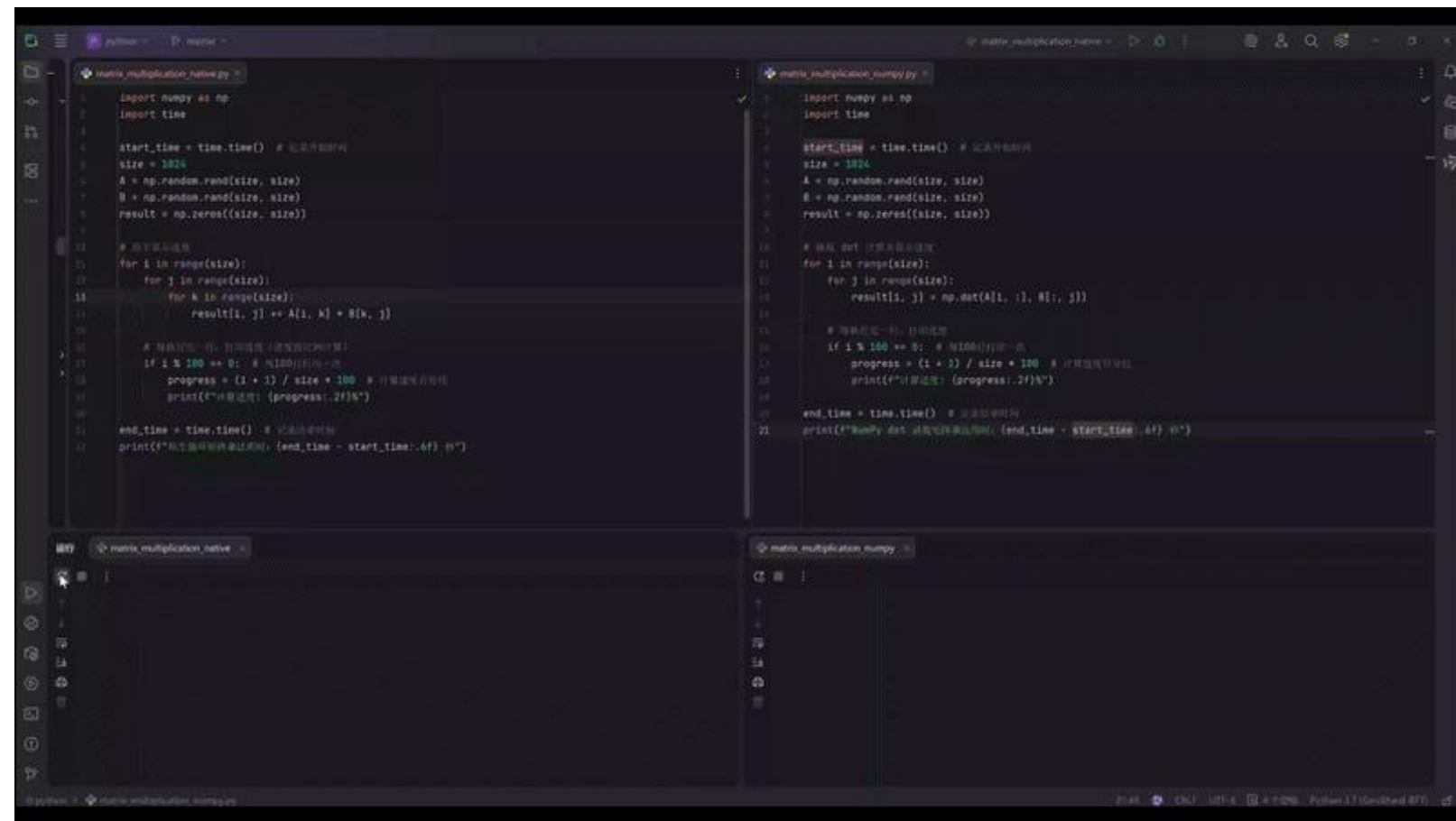
IC (指令数): 软件写得有多“废”。

CPI (每条指令周期): 硬件跑得有多“顺”。

Clock (时钟频率): 心脏跳得有多“快”。

性能差距：68倍的代价

同一个逻辑，表现差了**68**倍。你会雇佣那个让服务器电费翻**68**倍的程序员吗？



```
matrix_multiplication_native.py
1 import numpy as np
2 import time
3
4 start_time = time.time() # 记录开始时间
5 size = 1024
6 A = np.random.rand(size, size)
7 B = np.random.rand(size, size)
8 result = np.zeros((size, size))
9
10 # 三重循环实现
11 for i in range(size):
12     for j in range(size):
13         for k in range(size):
14             result[i, j] += A[i, k] * B[k, j]
15
16 # 每执行完一行，打印进度（进度条式计算）
17 if i % 100 == 0: # 每100行打印一次
18     progress = (i + 1) / size * 100 # 计算当前进度
19     print(f"进度: {progress:2f}%")
20
21 end_time = time.time() # 记录结束时间
22 print(f"原生循环耗时: {end_time - start_time:6f}s")

matrix_multiplication_numpy.py
1 import numpy as np
2 import time
3
4 start_time = time.time() # 记录开始时间
5 size = 1024
6 A = np.random.rand(size, size)
7 B = np.random.rand(size, size)
8 result = np.zeros((size, size))
9
10 # 使用 dot 计算乘积
11 for i in range(size):
12     for j in range(size):
13         result[i, j] = np.dot(A[i, :], B[:, j])
14
15 # 每执行完一行，打印进度
16 if i % 100 == 0: # 每100行打印一次
17     progress = (i + 1) / size * 100 # 计算当前进度
18     print(f"进度: {progress:2f}%")
19
20 end_time = time.time() # 记录结束时间
21 print(f"NumPy dot 耗时: {end_time - start_time:6f}s")

matrix_multiplication_native.py
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

matrix_multiplication_numpy.py
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

⚠ Python原生循环

408.75s

代码执行缓慢，消耗大量计算资源

🔧 MKL/CUDA优化

6.216s

调用底层库，充分利用硬件并行能力

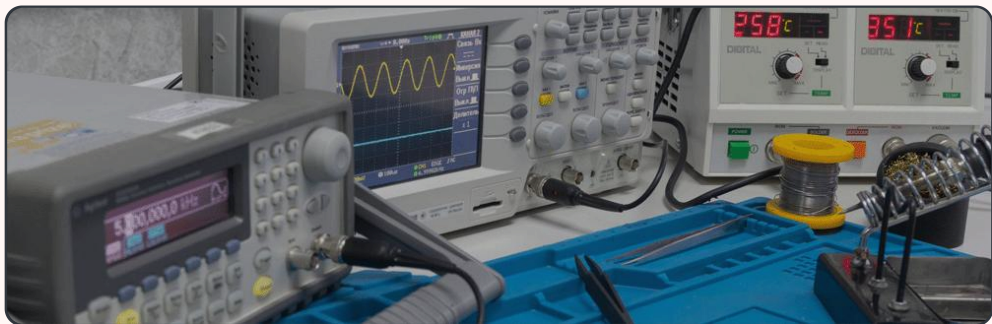
💡 课程核心价值

这门课教你的，就是如何**赚取这68倍的性能溢价**，成为能够挖掘硬件潜力的工程师。

目标：硬件感知型架构师

✗ 不是我们的方向

传统硬件工程师路线



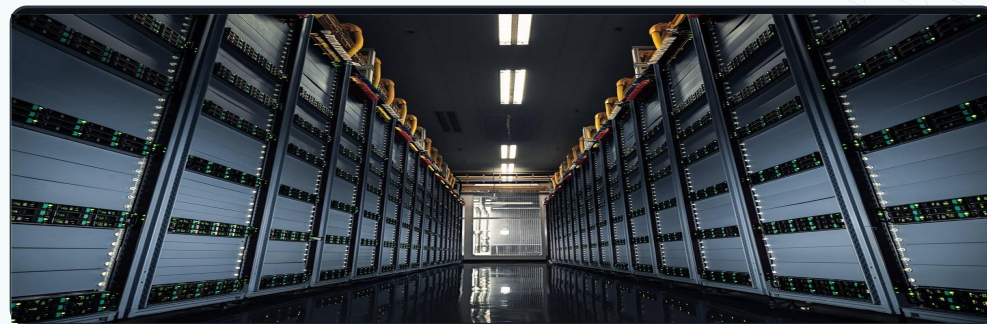
- ✗ 堆满焊枪、示波器的实验室
- ✗ 手工焊接电路板
- ✗ 复杂的门级电路设计
- ✗ 布满灰尘的调试环境



转型
目标

✓ 我们的目标

硬件感知型软件架构师



- ✓ 充满科技感的数据中心
- ✓ 高性能推理框架设计
- ✓ 自动驾驶芯片架构
- ✓ 系统级性能优化



核心能力目标

你能一眼看出系统的瓶颈在CPU核心、总线带宽还是显存容量

一个类比：盖房子 vs 住房子

🏗️ 建筑设计（体系结构）

房子有几层？几个房间？

门窗开在哪里？

水电管道怎么走？

→ 程序员能"看到"的接口

🔧 施工建造（组成原理）

用什么材料建？钢筋还是木头？

墙体多厚？地基多深？

如何保证抗震强度？

→ 硬件设计的实现细节

建筑领域	计算机领域	核心关注点
建筑设计蓝图	指令集架构 (ISA)	程序员可见的接口、功能定义
房间布局、门窗位置	寄存器、内存地址空间	数据和指令的组织方式
水电管线走向	总线、数据通路	信息的流动路径
施工材料与工艺	逻辑门、电路设计	硬件的物理实现
房屋抗震强度	可靠性、性能优化	系统的稳定性与效率

体系结构 vs 组成原理

对比维度	体系结构 (Architecture)	组成原理 (Organization)
关注点	程序员可见的抽象接口	硬件的具体实现方式
举例	是否有乘法指令？	乘法器用什么电路实现？
类比	汽车仪表盘（用户接口）	发动机内部结构
变化速度	变化较慢（兼容性）	每代都在优化



生活实例

Intel i7 和 i5 共享相同的 x86 体系结构（指令集相同），但组成不同（核心数、缓存大小不同）。就像两款不同配置的汽车，方向盘油门一样操作，但发动机不同。

授课内容

How a computer **is built**

- Logic->circuits->**datapath**

- **How a computer is controlled**

- Basic operations->micro architecture->instructions (ISA)->**assembly**

- **Contents (in-order)**

- MIPS/RISC-V assembly
- Logic design(adders,ALU,control)
- Performance analysis
- Data path and pipelining
- Input/Output
- Caches
- Virtual memory

授课内容

- Introduction
 - Chapters 1.1-1.6
- Principles of Quantitative Analysis; MIPS
 - Chapters 1.8-1.12
- Pipelining, Hazards, Implementation Issues
 - Chapters A.1-A.7, A.9, A.12, C.1-C.4
- Instruction-Level Parallelism (ILP), Tomasulo
 - Chapters C.5-C.9, 3.1, 3.2, 3.7
- Memory-Hierarchy Design, Cache Optimization
 - Chapters B.1-B.3, 2.1, 2.3
- ILP: Branch Prediction, Superscalars
 - Chapters 3.3-3.6, 3.8, 3.9
- Memory Hierarchy Design, Virtual Memory
 - Chapters 2.4, B.4
- Mostly covered by the textbook Appendix A, B, C and Chapter 1, 2, 3

教学目标

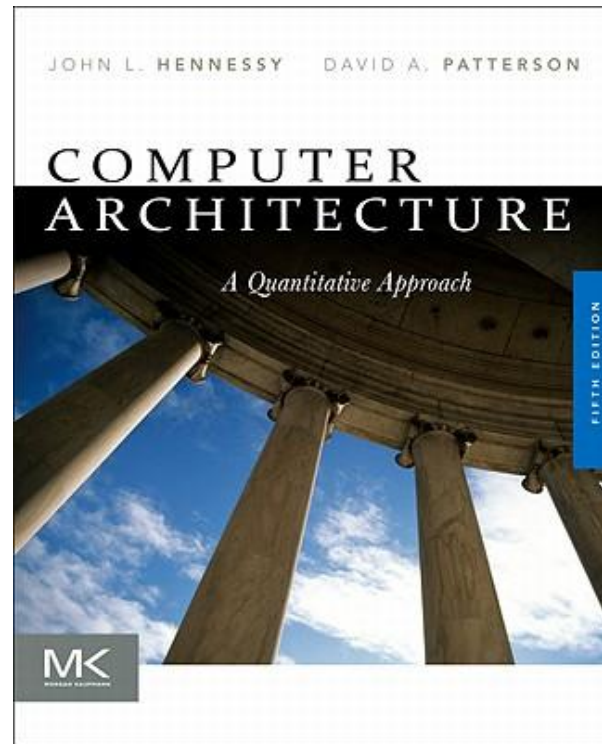
- Understand **how** modern computers **are designed and implemented**
 - computer architecture principles
 - design and implementation trade-offs
 - performance, cost, energy, design complexity
 - role of technology and market trends
 - not just microprocessor, but also servers and embedded systems
 - the arrival of multi-core architectures
 - case studies
 - not only classic processors, but also state-of-the art computers
- **“SW-oriented students”**
 - write better software applications
 - write better system’s software
 - operating systems, compilers

前置课程

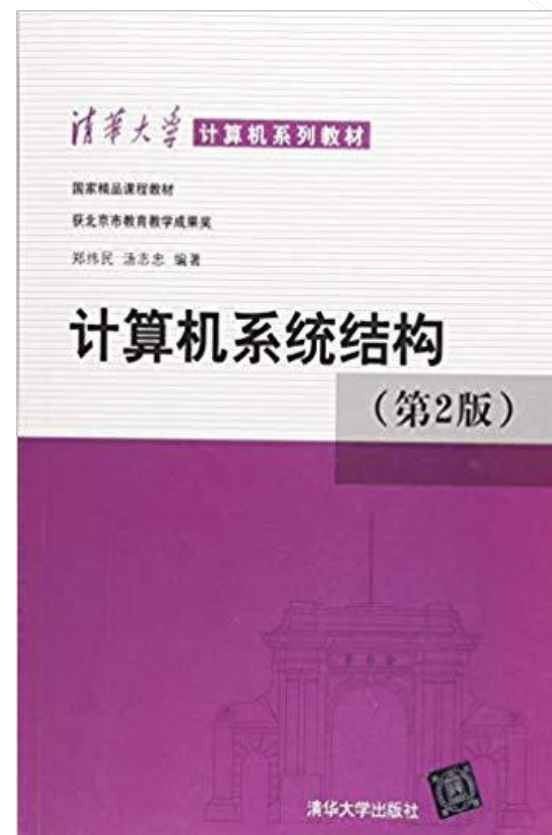
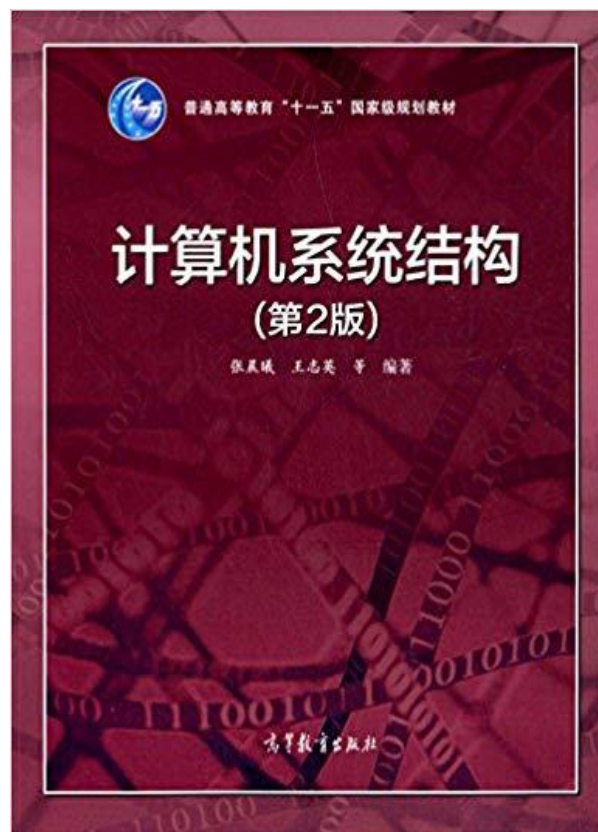
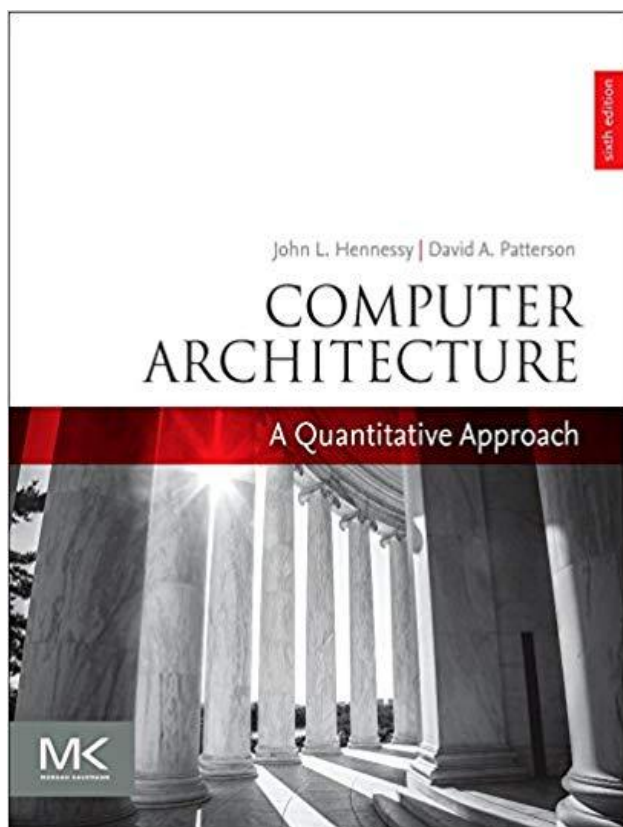
- **Course in computer organization**
 - processor organization, instruction set architecture
 - programming with an assembly language
 - ideally, basics of pipelining and caches
- **Course on digital logic**
 - logic gates, truth tables, Karnaugh maps
 - combinational vs. sequential logic, FSMs
- **Background in programming and data structure**
 - background in at least one high-level language
 - programming (C, C++, Java...)

参考教材

- Computer Architecture: A Quantitative Approach, 5th ed.," John L. Hennessy and David A. Patterson, Morgan Kaufman, 2011
- Additional readings will be assigned in class:
 - journal and conference papers
 - excerpts from other books
 - technical manuals



参考教材



参考教材

- D. Patterson and J. Hennessy,
“Computer Organization and
Design RISC-V Edition,”
Morgan Kaufmann, 2017
- D. Harris and S. Harris,
“Digital Design and Computer
Architecture, RISC-V Edition,”
Morgan-Kaufmann, 2021



课程评分

- **Grades** will be based on
 - Homework (about 15%)
 - In class quizzes (about 10%)
 - In class presentation (about 5%)
 - Midterm exam (about 10%)
 - Project / Experiments (about 20%)
 - Final exam (about 40%)
- Any material covered in class, assigned readings, handouts may appear in exam questions
- **Class Attendance:**
 - You are responsible for all material presented in class lectures
 - At times, lectures will diverge from the textbook
 - **Regular class attendance** is the best way to insure that you learn the material

课程评分

- Homework is due at the beginning of class on the due date
- Late policy
 - If you hand in something after the due date without the explicit written approval of an instructor or course TA, you might receive zero credit. In emergency only, contact the instructor in advance for an extension.
- General policy
 - Collaboration on solutions, or sharing or copying of solutions, is not allowed. No cooperation is allowed on exams. **This policy will be strictly enforced**

学习建议



理解优于记忆

不要死记公式，理解背后的"为什么"。比如 Cache 为什么有效？因为程序有局部性！



动手做实验

用模拟器亲手跑一遍流水线、Cache 命中，比看十遍书都有效



联系 AI 应用

每学一个知识点，想想它在 AI 系统中的角色。这让抽象概念变得生动



善用 AI 助教

遇到不理解的概念，可以问 ChatGPT/Claude 用类比帮你解释

联系方式

办公室：信息大楼B822
邮箱：liuduo@cqu.edu.cn

助教：

郭天泽，嘉陵江实验室，guotianze@stu.cqu.edu.cn

唐绎雄，嘉陵江实验室，tangyixiong@stu.cqu.edu.cn

钉钉扫码加入班级

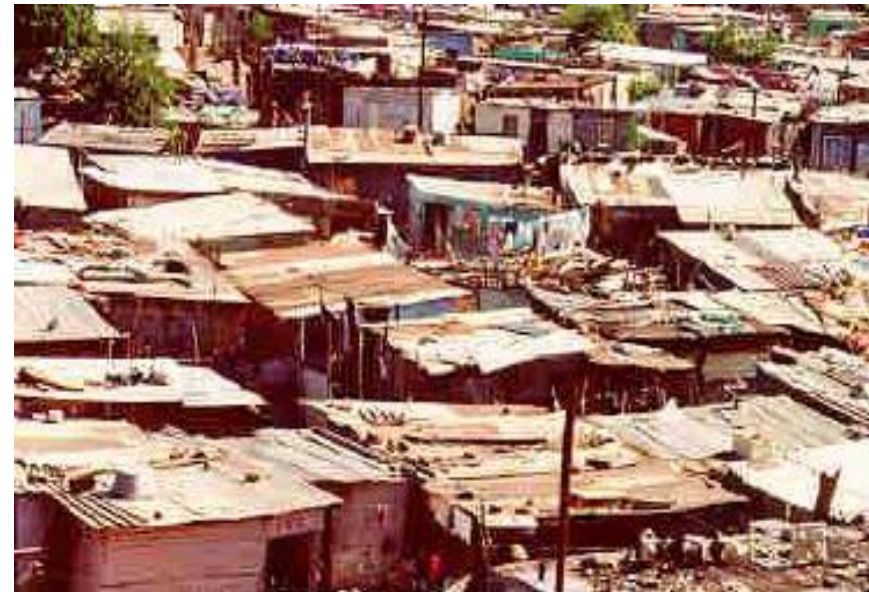


什么计算机体系结构?

- Computer architecture – The conceptual design and fundamental operational structure of a computer system.
 - CPU and Instruction Set
 - Access mode to memory
 - Components and their interconnection
 - ...



Habitat '67, by Moshe Safdie, at Montreal, Canada, 1967 ©
Artifice, Inc



[Sangiovanni-Vincentelli Vincentelli 04]

什么计算机体系结构？

- Computer Architecture is those aspects of the **instruction set** available to programmers, **independent of the hardware** on which the instruction set was implemented.
- The **term computer architecture** was first used in 1964 by Gene Amdahl, G. Anne Blaauw, and Frederick Brooks, Jr., the designers of the IBM System/360.
- The IBM/360 was **a family of computers** all with the same architecture, but with a variety of organizations (implementations).

计算机系统的抽象层次



应用程序层 (Application)

直接交互的软件：ChatGPT、抖音、英雄联盟、Office



高级编程语言层 (High-Level Language)

开发工具：Python, Java, C++, JavaScript



操作系统层 (Operating System)

资源管理与运行环境：Windows, Linux, macOS, Android



指令集架构层 (ISA) - 【本课程聚焦】

软硬件桥梁 (x86, ARM, RISC-V)：定义硬件能执行的指令集合



微体系结构层 (Microarchitecture) - 【本课程聚焦】

CPU内部实现 (流水线, Cache, 多核)：关注如何高效执行指令



数字逻辑/晶体管层 (Digital Logic) - 【先修知识】

物理基础：与门、或门、非门、CMOS晶体管

本课程聚焦

ISA 层：软硬件的桥梁

微架构层：CPU内部如何工作

先修知识

数字逻辑层

ISA：软件与硬件的“合同”



软件世界 (Software World) —— 点菜的顾客

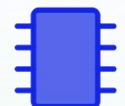
抽象指令集合，运行在硬件之上。包括：高级语言(Python/Java)、编译器、操作系统内核、应用程序。它们根据“菜单”发出指令。



ISA 指令集架构 —— 合同与菜单

软硬件协同工作的唯一接口。定义了软件能“点”什么（指令），以及硬件必须如何“上菜”（执行）。

规定内容：指令格式 · 数据类型 · 寄存器组 · 寻址方式 · 中断机制 · 特权级别



硬件世界 (Hardware World) —— 做饭的厨师

物理实现“合同”的电子电路。包括：CPU、内存(RAM)、总线、I/O设备。负责具体执行指令。

冯·诺依曼体系结构

三大核心思想

存储程序 (Stored Program)

程序和数据以二进制形式预先存储在存储器中，而非硬连线，赋予计算机通用性。

顺序执行 (Sequential Execution)

在控制器指挥下，自动连续地从存储器取指并执行，实现程序的自动化运行。

二进制 (Binary System)

所有信息（指令、数据）均以0和1表示，极大简化了硬件的物理实现。

五大基本组件架构

输入设备

键盘、鼠标等，将信息转换为二进制输入。

输出设备

显示器、打印机等，将结果转换为人可识别形式。

控制器 (CPU)

计算机的“指挥中心”，协调各部件有序工作。

运算器 (ALU)

计算机的“计算中心”，执行算术和逻辑运算。

存储器 (Memory)

存储程序和数据的桥梁。内存(RAM)速度快，外存(硬盘)容量大。。

“存储程序”为什么是革命性的？

以前：硬连线编程 (Wired Programming)

- ✓ **核心方式：**通过物理电路（电线、开关）定义程序，如 ENIAC。
- ✓ **巨大缺陷：**每换任务需重接线路，耗时数天，通用性极差。



就像换个电视频道需要拆开电视机，重新焊接内部线路一样麻烦。

现在：存储程序 (Stored Program)

- ✓ **核心思想：**程序与数据以二进制存储于内存，自动读取执行（冯·诺依曼结构）。
- ✓ **革命性优势：**同一硬件可加载不同程序，实现高效、灵活的通用计算。



就像智能手机，无需改装硬件，只需下载不同的App即可实现无数功能。

革命性影响总结

- 实现了计算机的真正通用化，奠定现代计算机基础。
- 催生了独立的软件产业，使功能扩展无限可能。
- AI视角：正因如此，今天的GPU既能训练ChatGPT，也能跑自动驾驶模型——只需加载不同程序！

课堂思考：架构与技术的演进



核心问题：你的手机具备 CPU、内存和存储，它是冯·诺依曼机吗？如果是，它与 1946 年的 ENIAC 有何根本区别？

01 架构层面：五大部件

- **对应关系：**触摸屏(输入)、显示屏(输出)、SoC(运算/控制)、RAM/闪存(存储)。
- **结论：**现代手机依然严格遵循冯·诺依曼体系结构。



02 性能层面：指数级跃升

- **ENIAC vs 手机：**从每秒5000次运算到数万亿次，重量从30吨降至几十克。
- **结论：**性能提升亿倍，但存储程序、顺序执行的原理未变。



03 技术层面：物理革命

- **演进路径：**电子管 → 晶体管 → 集成电路 → 纳米级芯片。
- **结论：**架构思想基本不变，但半导体实现技术发生了天翻地覆的革命。



总结与升华：架构的生命力与技术的驱动力

冯·诺依曼架构历经80年仍为设计基石，体现了优秀架构的强大生命力；而计算机性能的指数级增长，主要归功于半导体工艺的突破。理解经典架构，是探索未来（如量子计算）的基础。

异构计算：big.LITTLE 架构解析

核心概念：异构双核集成

- **Big Cores (大核):** 追求极致性能，处理高负载任务（如游戏、渲染）。
- **Little Cores (小核):** 追求极致能效，处理轻负载任务（如待机、后台）。

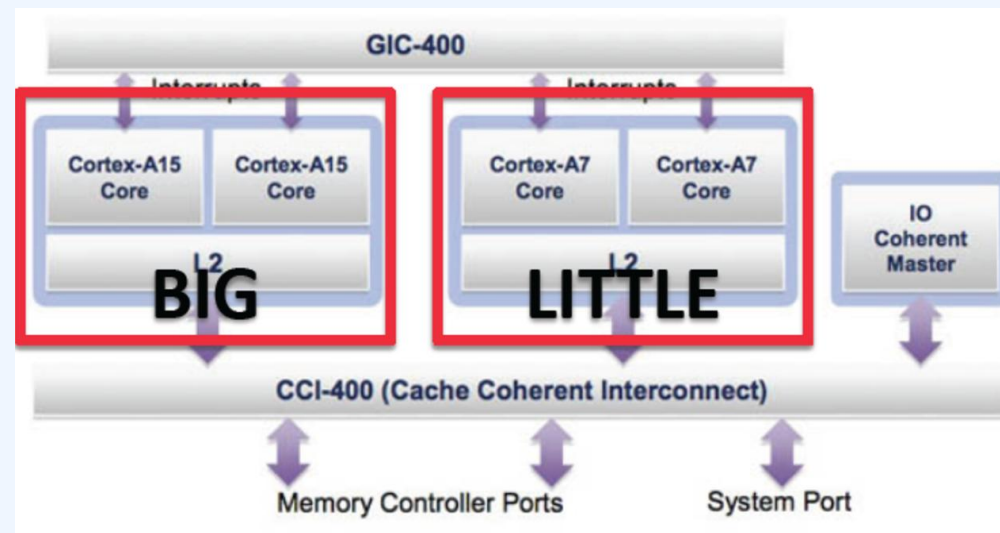
架构优势：智能调度与平衡

- **按需分配:** 系统自动将任务分配给最适合的核心，拒绝资源浪费。
- **续航提升:** 轻负载时仅小核工作，大幅降低功耗，延长电池寿命。

应用场景：从移动到云端

- **移动设备:** 智能手机与平板的标配，平衡性能与发热。
- **扩展领域:** ARM 轻薄本（如Apple Silicon）及数据中心服务器。

典型 big.LITTLE 架构示意图



Big 集群 (高性能) 与 Little 集群 (高能效)
通过 CCI (Cache Coherent Interconnect) 互联
共享内存与 I/O 资源，由系统统一调度

总结：big.LITTLE 是异构计算的里程碑，通过组合不同核心，实现了**性能与功耗的最佳平衡**

能效比的艺术：ARM 为何选择 big.LITTLE?

Why is ARM Doing This? —— 从能效比看异构架构的必然性

核心公式： $\text{Power Efficiency} = \text{Calculations} / \text{Energy}$ (能效比 = 计算量 / 消耗的能量)

这是衡量处理器设计优劣的黄金标准：用最少的能量完成最多的计算。

Why is ARM Doing This?

- **Power Efficiency** = calculations/energy



big.LITTLE 解决方案

- **核心理念：**“让合适的核心做合适的事”，智能调度。
- **系统收益：**组合使用大小核，避开单一核心的能效低谷，始终运行在全球最优曲线上。

核心总结：突破物理极限，兼顾性能与续航

ARM 采用异构架构的根本原因是为了突破单一核心的能效比极限。通过动态调度，完美解决了移动设备对高性能和长续航的双重需求，这已成为现代处理器设计的行业标准。

架构细节：大小核的微体系结构差异

LITTLE Core (小核)

设计目标：极致能效 (低功耗)

简单结构 & 短流水线

功能单元少，指令步骤少，时钟频率较低

顺序执行 (In-order)

严格按序执行，硬件简单，能效比极高

适用场景

轻负载任务（如待机、浏览），省电首选

BIG Core (大核)

设计目标：极致性能 (高性能)

复杂结构 & 长流水线

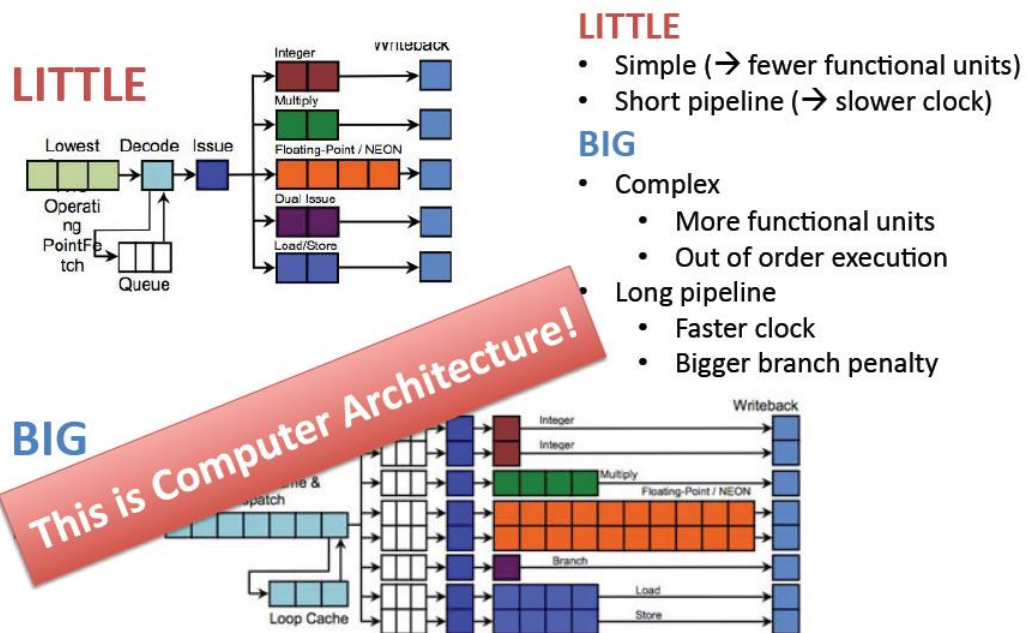
更多ALU/FPU单元，高时钟频率，大缓存

乱序执行 (Out-of-order)

动态调整指令顺序，挖掘指令级并行性(ILP)

微体系结构对比

The Details

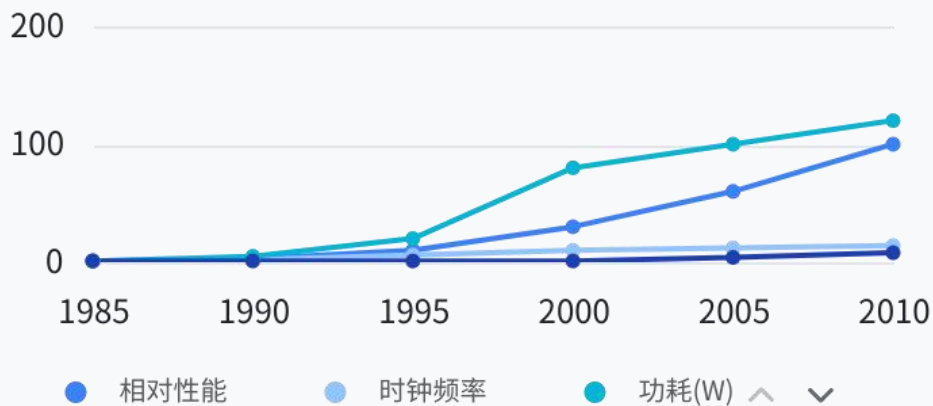


架构总结：大小核的差异是“性能”与“功耗”的权衡 (Trade-off)，big.LITTLE 将两者有机结合实现最佳平衡。

功耗墙 (Power Wall)：架构创新的催化剂

Why Should They Do This? —— 为何必须从“提升性能”转向“提升能效比”？

Power Wall 趋势分析 (1985-2010)



- **功耗墙显现 (2000后):** 功耗增长趋缓，无法再通过高频换取性能。
- **架构转型:** 核心数（深蓝线）开始激增，转向多核并行。

为何出现“功耗墙”？

- **物理极限:** 晶体管尺寸与电压存在下限，漏电效应显著。
- **散热挑战:** 功耗增加导致发热剧增，散热能力逼近极限。
- **能效困境:** 单纯提升频率导致功耗呈指数级增长 ($P \approx CV^2f$)。

破局之道：提升能效比 (Performance/Watt)

- **异构计算:** 如 big.LITTLE 架构，组合不同核心优化能效。
- **专用加速:** 为 AI/图形等特定任务设计专用硬件。
- **精细管理:** 独立电源域控制，按需分配算力。

总结: 功耗墙标志着计算机架构从“唯性能论”进入“能效优先”的新时代，是推动多核与异构计算发展的根本动力。

异构计算的多样性：GPU的众核并行之路

GPU 设计哲学：Lots of Small Cores

- ✓ 极致吞吐量 (Throughput): 牺牲单线程延迟，追求海量并行计算能力。
- ✓ SIMT 单指令多线程: 同一指令流控制成百上千线程，适合矩阵运算。
- ✓ 简化核心结构: 去除复杂的乱序执行单元，集成更多流处理器。

CPU vs GPU 架构对比

特性	CPU (big.LITTLE)	GPU (众核并行)
核心数量	少 (2-8大核+小核)	极多 (成百上千流处理器)
设计目标	低延迟、通用性	高吞吐量、并行计算
擅长任务	复杂串行、操作系统	图形渲染、AI矩阵运算

经典 GPU 架构解析

NVIDIA Fermi 架构 (GPGPU 里程碑)

- 核心组件：流式多处理器 (SM)
每个SM包含多个CUDA核心，奠定了通用计算基础，强调高带宽内存访问。

AMD Barts 架构 (并行计算单元)

- 核心组件：流处理器簇 (SIMD Engines)
展示了由大量计算阵列组成的特点，通过阵列化设计实现海量数据吞吐。

💡 面对“功耗墙”，GPU选择了与CPU截然不同的路径：通过众核并行架构，在专用领域实现极致能效比，共同构建了现代异构计算生态。

为何学习计算机体系结构？ (So, Why Should You Care?)

无论你是程序员、算法工程师还是产品经理，理解体系结构都将为你带来独特的视角和竞争优势。



技术演进：从单核到量子计算

硬件架构创新日新月异（CPU/GPU/TPU/量子计算），摩尔定律虽放缓，但架构创新仍驱动算力指数级增长。不理解底层演进，就无法把握上层应用趋势，易被技术浪潮淘汰。



透过现象：理解技术变革的本质

技术变革源于底层约束（如功耗墙催生big.LITTLE，AI爆发依赖GPU众核架构）。掌握体系结构能让你看清变革的根本驱动力，从而做出更具前瞻性的技术决策。



性能基石：效率与竞争力的核心

代码性能不仅取决于算法复杂度，更依赖于对缓存、并行等硬件特性的利用。理解体系结构是写出高性能代码、设计优秀产品的前提。



软硬协同：跨越壁垒的系统视角

体系结构是软硬件的桥梁。从OS调度策略到算法设计（如CUDA编程），理解它能打通软硬件壁垒，让你在更广阔的空间内进行系统级优化与创新。

案例分析：如何选择合适的处理器？

选择处理器是一个基于具体应用场景的多目标优化与权衡过程。



场景一：智能手机处理器选择

核心目标：能效比优先，兼顾集成度

关键考量：ARM big.LITTLE异构设计，集成GPU/ISP/NPU，严格控制BOM成本。

典型选择：Qualcomm Snapdragon, Apple A-series, Samsung Exynos。

场景二：数据中心服务器处理器选择

核心目标：高吞吐与并行性，兼顾可靠性

关键考量：多核多线程(x86主流)，支持ECC内存与高MTBF，注重长期运营能效(TCO)。

典型选择：Intel Xeon, AMD EPYC, NVIDIA A100。

总结：处理器选择没有通用标准，场景决定权重。手机追求极致能效集成，服务器追求高并发与可靠性。

中国超级计算机发展成就一览



全球领先地位

多台系统位居世界前列，
算力规模持续扩大



算力性能卓越

Linpack性能突破十亿亿
次/秒，算力强劲



应用领域广泛

覆盖气象、生物、航天
及互联网大数据分析



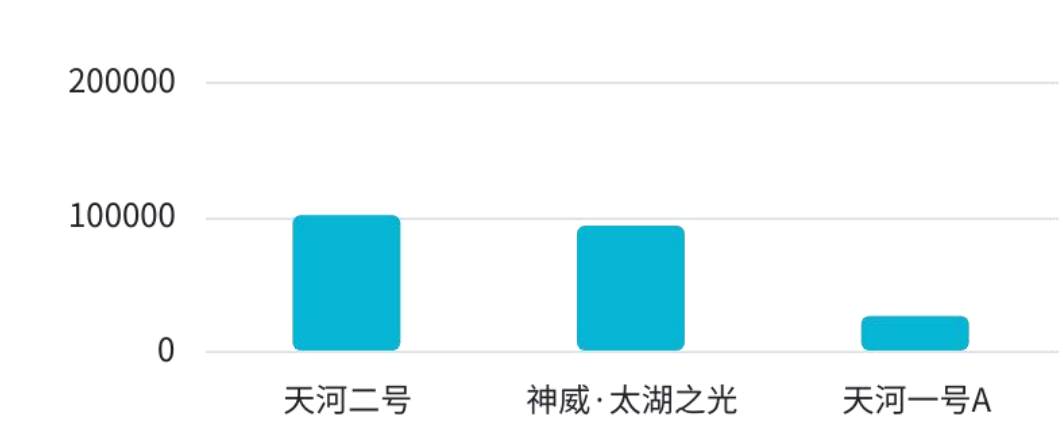
技术路线多元

自主申威架构与国际合
作架构并行发展

核心超算系统性能数据

排名	型号	年份	Linpack (PFLOPS)
1	天河一号	2009	2.57
2	天河二号	2013	10.07
3	神威·太湖之光	2016	12.5

典型超算性能对比 (TFLOPS)



神威·太湖之光：自主可控的典范

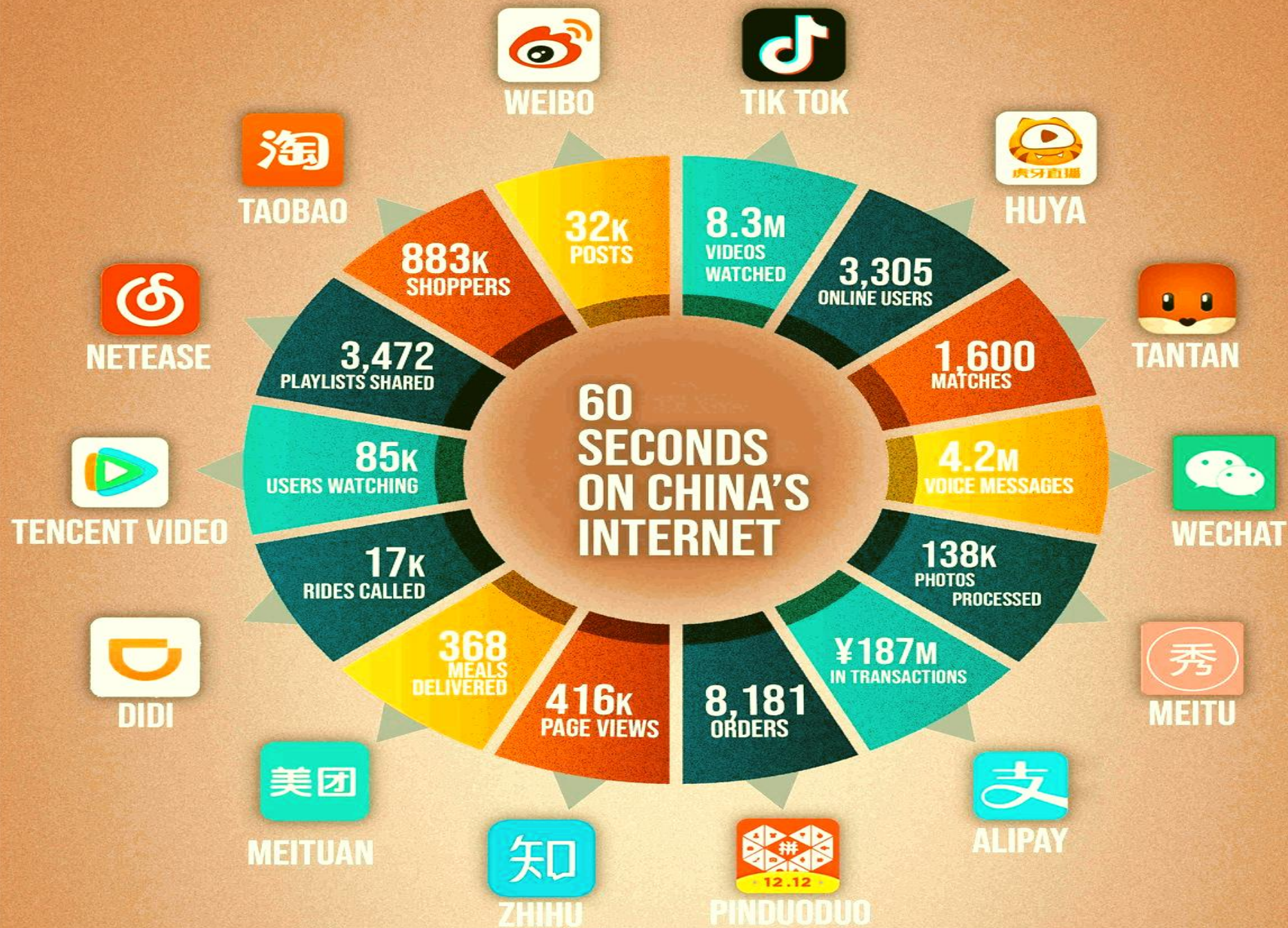
采用自主研发的“申威26010”众核处理器，不依赖国外技术。
应用于气候气象、生物医药等领域，曾多次蝉联全球超算
TOP500榜首。



天河二号：异构计算的标杆

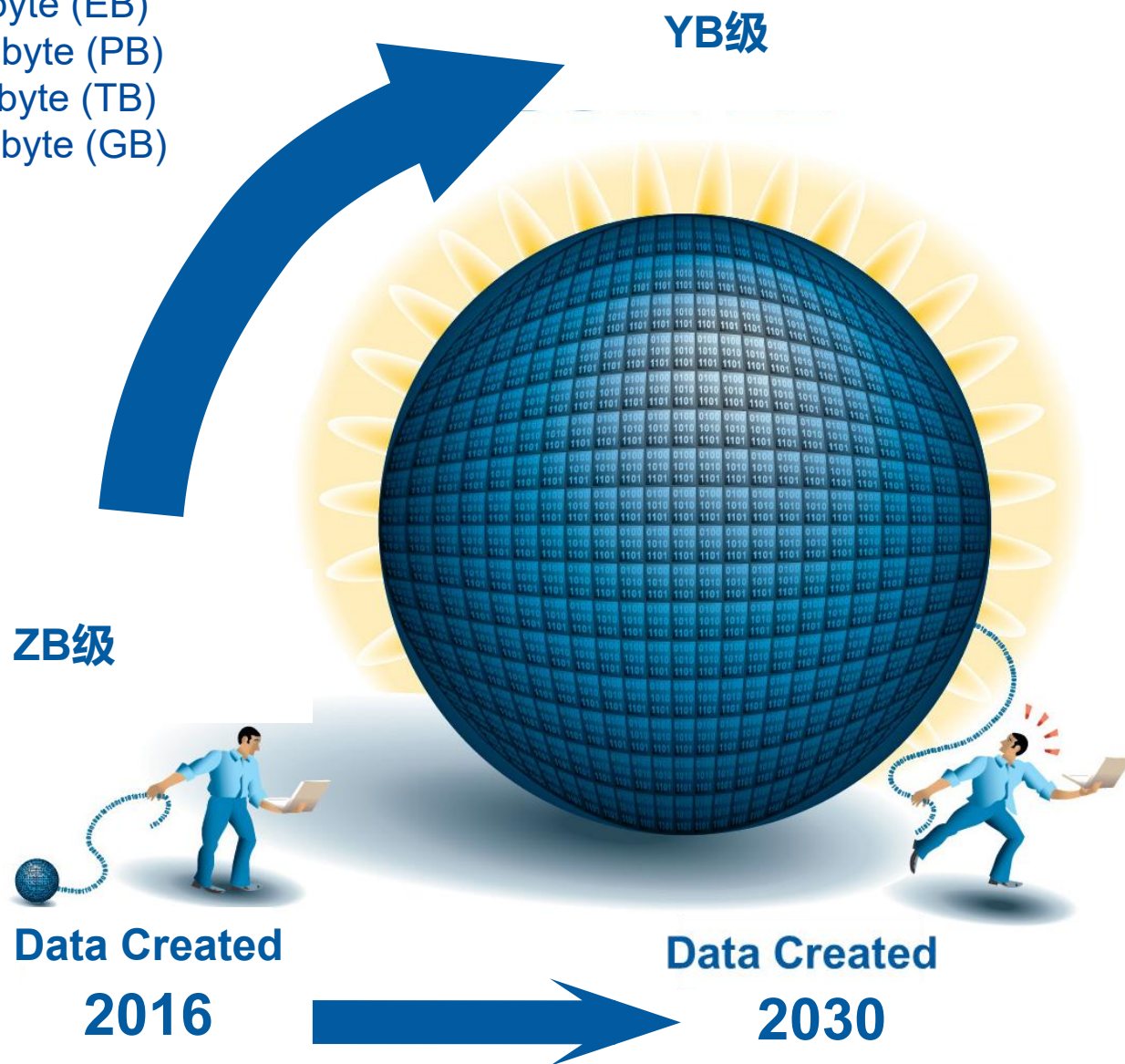
采用异构架构。长期位居全球前列，为国家重大科技项目
和产业创新提供强大算力支撑。

中国互联网 60秒



Data Explosion

1 ZB = 1,024 Exabyte (EB)
1 EB = 1,024 Petabyte (PB)
1 PB = 1,024 Terabyte (TB)
1 TB = 1,024 Gigabyte (GB)



公制前綴：量化世界的語言

Metric prefixes							
Prefix	Symbol	1000^m	10^n	Decimal	Short scale	Long scale	Since ^[n 1]
yotta	Y	1000^8	10^{24}	1 000 000 000 000 000 000 000 000 000	septillion	quadrillion	1991
zetta	Z	1000^7	10^{21}	1 000 000 000 000 000 000 000 000	sextillion	trilliard	1991
exa	E	1000^6	10^{18}	1 000 000 000 000 000 000 000	quintillion	trillion	1975
peta	P	1000^5	10^{15}	1 000 000 000 000 000 000	quadrillion	billiard	1975
tera	T	1000^4	10^{12}	1 000 000 000 000 000	trillion	billion	1960
giga	G	1000^3	10^9	1 000 000 000	billion	milliard	1960
mega	M	1000^2	10^6	1 000 000	million		1960
kilo	k	1000^1	10^3	1 000	thousand		1795
hecto	h	$1000^{2/3}$	10^2	100	hundred		1795
deca	da	$1000^{1/3}$	10^1	10	ten		1795
		1000^0	10^0	1	one		–

公制前綴：量化世界的語言

Metric prefixes							
Prefix	Symbol	1000^m	10^n	Decimal	Short scale	Long scale	Since ^[n 1]
		1000^0	10^0	1	one		—
deci	d	$1000^{-1/3}$	10^{-1}	0.1	tenth		1795
centi	c	$1000^{-2/3}$	10^{-2}	0.01	hundredth		1795
milli	m	1000^{-1}	10^{-3}	0.001	thousandth		1795
micro	μ	1000^{-2}	10^{-6}	0.000 001	millionth		1960
nano	n	1000^{-3}	10^{-9}	0.000 000 001	billionth	milliardth	1960
pico	p	1000^{-4}	10^{-12}	0.000 000 000 001	trillionth	billionth	1960
femto	f	1000^{-5}	10^{-15}	0.000 000 000 000 001	quadrillionth	billiardth	1964
atto	a	1000^{-6}	10^{-18}	0.000 000 000 000 000 001	quintillionth	trillionth	1964
zepto	z	1000^{-7}	10^{-21}	0.000 000 000 000 000 000 001	sextillionth	trilliardth	1991
yocto	y	1000^{-8}	10^{-24}	0.000 000 000 000 000 000 000 001	septillionth	quadrillionth	1991

1. ^ The metric system was introduced in 1795 with six prefixes. The other dates relate to recognition by a resolution of the CGPM.

数据存储单位的演进

核心洞察与演进

层级体系

从Byte开始，按1024进制递进，形成庞大的存储金字塔。

二进制标准

计算机领域标准，1KB = 1024 Bytes，精确反映实际存储。

提示:这是硬盘“缩水”现象的根源

十进制简化

商业宣传常用，1KB = 1000 Bytes，便于理解记忆。

海量数据时代

PB、EB、ZB已成为描述数据中心和全球数据总量的常用单位。

- ❑ **Kilobyte** – 2^{10} or 1,024 bytes
- ❑ **Megabyte** – 2^{20} or 1,024 Kilobytes
 - / sometimes “rounded” to 10^6 or 1,000,000 bytes
- ❑ **Gigabyte** – 2^{30} or 1,024 Megabytes
 - / sometimes rounded to 10^9 or 1,000,000,000 bytes
- ❑ **Terabyte** – 2^{40} or 1,024 Gigabytes
 - / sometimes rounded to 10^{12} or 1,000,000,000,000 bytes
- ❑ **Petabyte** – 2^{50} or 1024 Terabytes
 - / sometimes rounded to 10^{15} or 1,000,000,000,000,000 bytes
- ❑ **Exabyte** – 2^{60} or 1024 Petabytes
 - / Sometimes rounded to 10^{18} or 1,000,000,000,000,000,000 bytes
- ❑ **Zettabyte** – 2^{70} or 1024 Exabytes
 - / Sometimes rounded to 10^{21} or 1,000,000,000,000,000,000,000 bytes

天河三号：中国自主研发的E级超算原型机

 全面搭载国产自主芯片，实现核心技术自主可控；自主研发操作系统与并行开发系统，构建完整技术体系；峰值性能达百亿亿次，代表世界顶尖水平。



国产自主芯片

摆脱国外依赖，保障信息安全，核心技术自主可控



自主操作系统

深度优化硬件性能，提供高效稳定的运行环境



并行开发系统

支持大规模并行计算，提供强大的软件支撑能力



关键性能指标

峰值计算能力

100 ExaFLOPS

系统特性

高可靠性设计 / 高效冷却



应用前景与战略意义

科学与工程

气象预报、航空航天、能源勘探

产业与安全

AI大数据、云计算、国家安全保障

神威·太湖之光：超算应用的璀璨成果

依托自主研发的“申威26010”众核处理器，“神威·太湖之光”在气象、航空航天、生物医药等领域展现了强大的通用计算能力，为国家科技创新和产业升级提供了坚实保障。

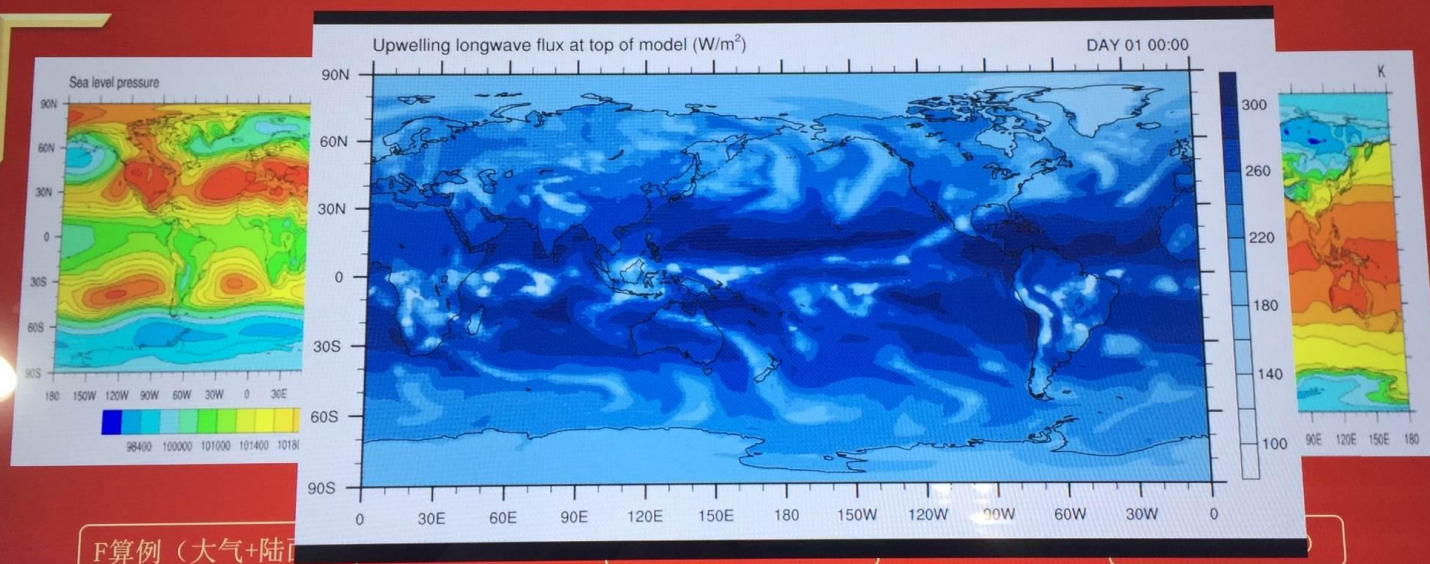
“神威·太湖之光”应用课题列表							
序号	领域/课题名称	承担单位	规模	序号	领域/课题名称	承担单位	规模
1	气象 全球大气非静力云分辨模拟	中科院软件所、清华大学等	整机	16	物理 Crystal-MD分子动力学模拟	北京科技大学	半机
2	海洋 高分辨率海浪数值模式	国家海洋局海洋一所	整机	17	能源 三维波动方程正演模拟	中国地质大学（武汉）	半机
3	材料 钛合金微结构演化相场模拟	中科院计算机网络信息中心	整机	18	航空 湍流的直接数值模拟	北京大学	半机
4	生物 骆驼基因组功能注释及同源基因家族构建	上海科学院上海生物信息技术研究中心	整机	19	航天 高超声速内外流数值模拟	中国空气动力研究发展中心	32768核
5	航天 航天飞行器全流域数值模拟	国家计算流体力学实验室	整机	20	气象 GRAPES全球数值预报模式	中国气象局	16384核
6	核聚变 托克马克磁约束核聚变装置中的逃逸电子轨迹模拟	中国科技大学核学院	整机	21	气候 地球系统模式	清华大学、北京师范大学	16384核
7	航天 可压缩边界层湍流及转捩模拟	中科院力学所	半机	22	航空 航空飞行器模拟程序	清华大学航空航天学院	16384核
8	生物 蛋白质分子动力学模拟	北京大学	半机	23	航空 飞机、高速列车流场分析	中科院深圳先进技术研究院	10000核
9	电磁 FDTD时域有限差分法电磁模拟软件	西安电子科技大学	半机	24	气象 城市污染扩散	中科院深圳先进技术研究院	10000核
10	电磁 PO物理光学法电磁模拟软件	西安电子科技大学	半机	25	生命科学 脑血流模拟	中科院深圳先进技术研究院	10000核
11	船舶 潜艇声学特性模拟计算	中船重工集团702所	半机	26	生物 NIMD分子动力学计算	上海药物所	8192核
12	船舶 大型海上浮动平台波浪载荷计算	中船重工集团702所	半机	27	航空 飞机外型全流场优化设计	631所	8192核
13	动漫 真实感动漫渲染	山东大学	半机	28	航空 大型客机减阻与激波控制	南京航空航天大学	8192核
14	核模拟 低能段GMNT软件	中科院近代物理研究所	半机	29	航空 航空发动机高精度数值模拟	浙江大学	8192核
15	生物 测序数据精确对比	山东大学	半机	30	能源 风电场设计评估云服务平台	远景能源公司	4096核

神威·太湖之光：超算应用的璀璨成果

地球系统模式

清华大学地学中心

国内首次实现**百万核**规模的全球**10公里**分辨率地球系统模式数值模拟，全面提高我国应对极端气候事件和自然灾害的减灾防灾能力，为我国参与国际气候环境外交谈判，提升我国在气候变化领域国际影响力提供重要基础

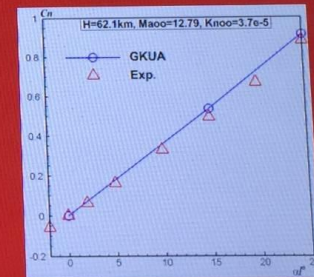
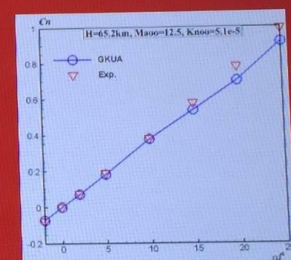
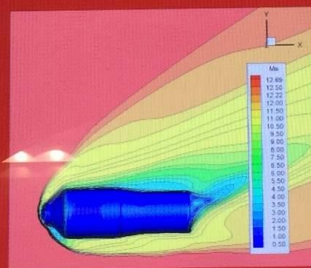
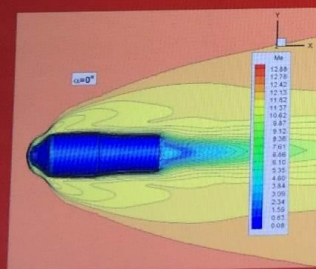
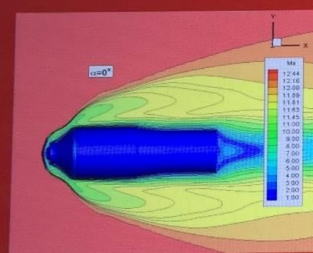


神威·太湖之光：超算应用的璀璨成果

航天飞行器统一算法数值模拟

国家计算流体力学实验室

完成**天宫-1**飞行器两舱简化外形(长度10余米、横截面直径近3.5米)陨落飞行 $H=65\text{km}$ 、 62km 、 $Ma=13$ 绕流状态大规模并行计算,使用16384个处理器在**20天**内完成常规需要**12个月**的计算任务,计算结果与风洞实验结果符合较好。



天河二号：曾经的世界超算之巅



 峰值性能

5.49 亿亿次/秒

 持续性能

3.39 亿亿次/秒

 能效比

1.931 GFlops/W

 总核心数

>300万 颗

 研发战略意义

- 技术积累：培养了大批专业人才，积累了宝贵研发经验
- 产业带动：推动高性能计算产业链升级与技术进步
- 国际影响：提升了中国在超算领域的国际地位与话语权

计算机体系结构：从“旧”视角到“真实”定义

定义演变：从单一视角到系统工程



“旧”视角：聚焦 ISA 设计

早期仅关注指令集架构（寄存器、寻址等），视角相对狭窄。



“真实”定义：综合系统工程

涵盖 ISA、微架构、硬件实现，在成本、功耗、可用性约束下追求性能最大化。

核心价值与前沿趋势



性能与效率的基石

决定计算机运算速度、存储能力及处理效率，是系统优化的核心。



前沿研究方向

聚焦异构计算、能效优化、安全可靠等领域，推动技术持续创新。



核心研究内容

- **ISA与微架构**：定义软硬件接口，设计流水线、缓存等硬件结构。
- **性能与能效**：通过并行技术提升性能，在保证性能的前提下降低功耗。
- **可靠性与安全**：保障系统稳定运行，防止硬件故障与恶意攻击。



设计原则与重要性

- **权衡与平衡**：在性能、功耗、成本之间寻求最佳平衡点。
- **生态与创新**：ISA决定软件生态，架构创新推动产业技术发展。
- **产业支撑**：先进架构是信息技术产业竞争力的基石。

计算机体系结构：从狭义到广义的定义与协同设计

定义范围演进：从狭义到广义

■ 狭义定义 (Narrow Definition)

聚焦指令集架构(ISA)与微架构，关注硬件组织与实现方式，侧重底层逻辑。

■ 广义定义 (Broad Definition)

涵盖算法、程序、子系统、电路及电子层级，强调系统整体性与跨层优化。

→ 软硬件协同设计 (Co-design)



性能最大化

软硬配合释放硬件潜力



能效最优平衡

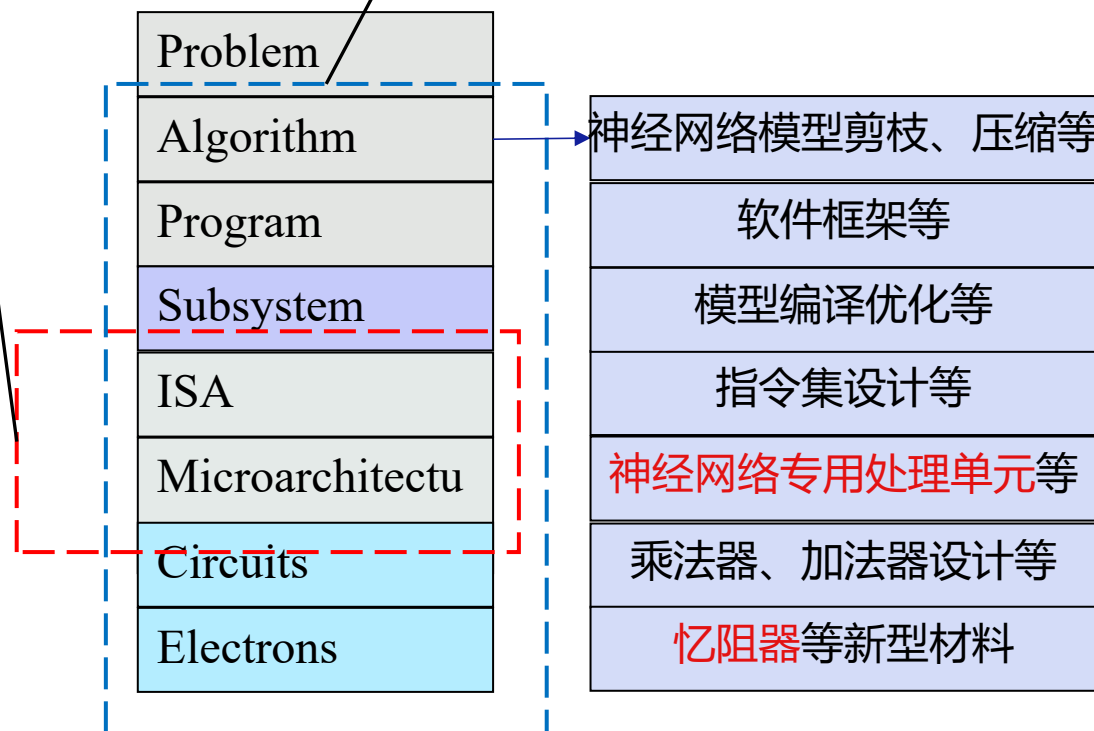
针对性优化降低功耗



典型应用： GPU加速计算、AI专用芯片(TPU)
框架适配

Narrow Definition

Broad Definition



软硬件协同设计

技术进步：从工具革新到社会变革



从单一设备到智能终端的演进

集成了相机、收音机、电话等多种功能，体现了技术的高度集成与便利化。

影响与未来趋势

- 🧠 **人工智能**：普及应用，重塑生产生活方式
- 🌐 **物联网**：万物互联，构建智能化环境
- 🔢 **量子计算**：突破算力极限，解决复杂难题
- 🌱 **绿色技术**：推动可持续发展，实现生态友好



技术趋势：从数据增长到创新突破

核心技术增长趋势概览

集成电路整体集成度：年增长率约**40-55%**

闪存容量增长：年增长率约**50-60%**

关键技术指标解读

晶体管密度与芯片尺寸

密度年增35%，芯片面积年扩10-20%，系统级集成度显著提升。

存储技术差异化发展

闪存增长最快(50-60%)，磁盘技术凭借低成本优势保持40%增长。

驱动因素分析

- 摩尔定律延续：计算性能持续提升，每18-24个月翻倍。
- 市场需求牵引：大数据与AI对存储容量和算力提出更高要求。
- 成本效益驱动：规模效应与技术创新降低单位成本，提升竞争力。

未来发展展望

- 突破物理极限：新材料与架构创新是延续增长的关键。
- 新兴技术崛起：量子计算与AI芯片将重塑计算格局。
- 绿色可持续：提升能效与降低碳排放成为核心发展方向。

带宽与延迟：性能发展的不平衡性与优化路径

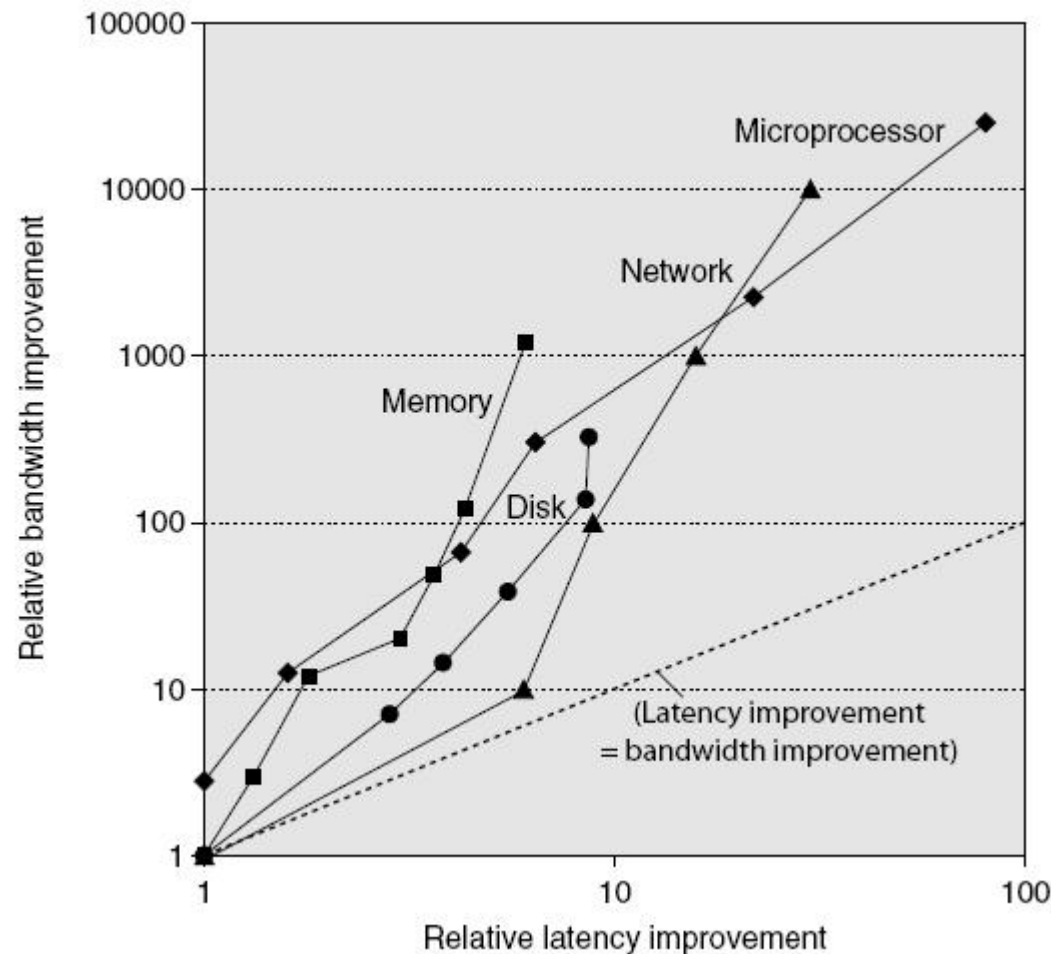
🔊 总结：带宽与延迟发展不平衡，延迟的相对缓慢提升已成为制约系统实时性的关键瓶颈，未来优化将更聚焦于响应速度的提升。

📊 带宽与延迟发展趋势对比

- 带宽提升：处理器(10k-25k倍) > 内存/磁盘(300-1200倍)
- 延迟改进：处理器(30-80倍) > 内存/磁盘(6-8倍)
- 趋势解读：对数坐标下，带宽曲线斜率远大于延迟，差距持续扩大。

🔍 性能差异成因分析

- **物理限制**：光速、电子传输速度等基本物理定律限制了延迟的降低。
- **技术瓶颈**：传统存储介质（如机械硬盘）在寻道时间上难以突破。
- **系统复杂性**：多层级架构增加了数据路径，导致延迟来源多样化。

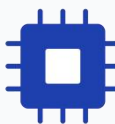


多核片上系统 (SoC) : 架构多样性与设计挑战



核心概念解析

多核SoC通过集成多核心提升并行处理能力。同构多核设计简单, 适合通用计算; 异构多核结合大核与小核, 在移动设备中实现性能与能效的最佳平衡。

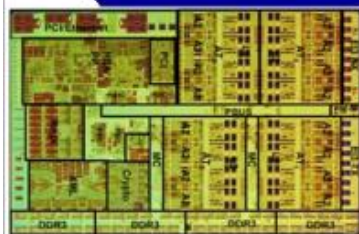


多核SoC架构概览

集成多个处理器核心 (同构或异构), 实现单芯片上的高性能计算。架构从传统SMP向big.LITTLE等异构模式演进, 核心间通信与资源共享是关键。

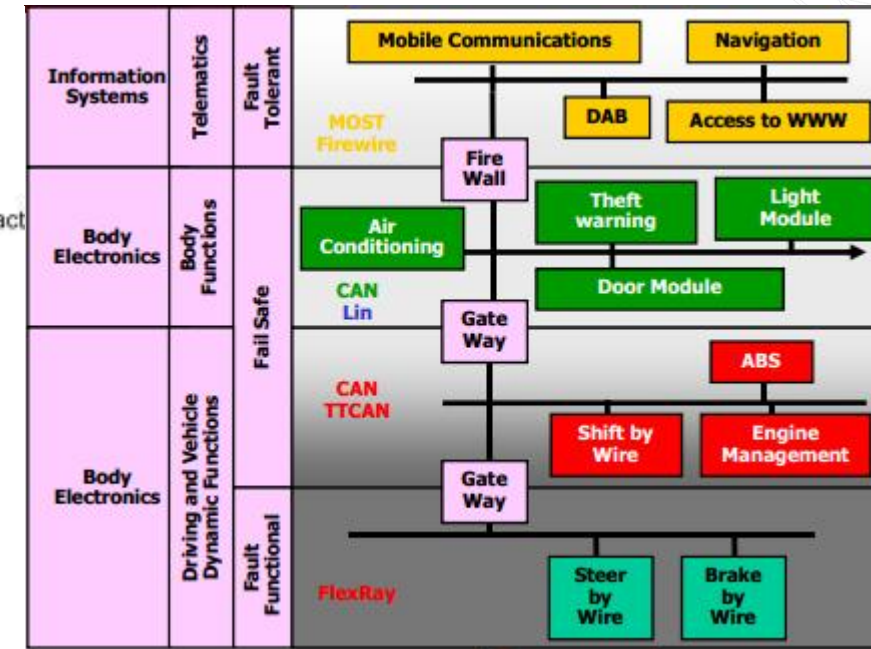
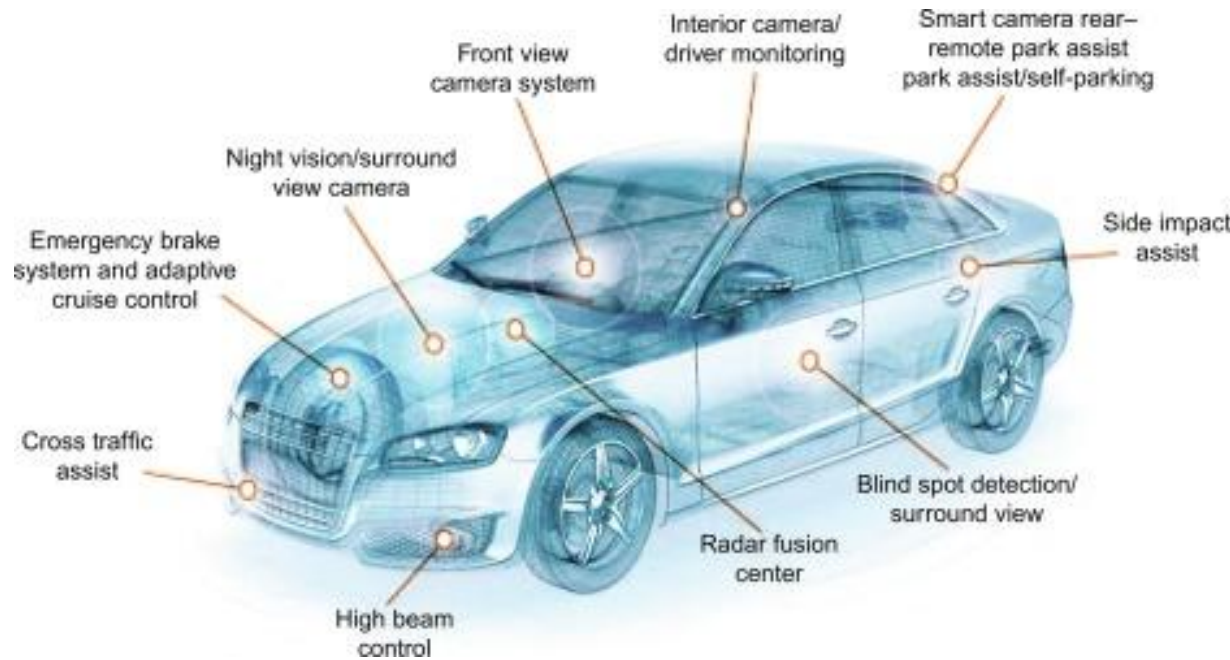


- A rich variety of multi-core architectures, and growing...
 - shift from homogeneous tile-based chip multi-processors (CMP) to heterogeneous multi-core systems-on-chip (SOCs)
- Complexity of design and programming
 - increasing number of heterogeneous cores
 - high-performance design is *power-efficient* design
 - resiliency to parameter variations, component faults,...
 - need for a new HW/SW interface
 - increasing impact of communication



异构嵌入式系统 (Heterogeneous Embedded Systems)

I Electronics for the Car



- Up to 70 Electronic Computing Units (ECUs) in a modern car like a BMW Series 7
 - Heterogeneous communication networks
 - DSC (dynamic stability control) contains ABS as one of 15 sub-functionalities

当前体系结构趋势

- Cannot continue to leverage Instruction-Level parallelism (ILP)
 - Single processor performance improvement ended in 2003
- New models for performance:
 - Data-level parallelism (DLP)
 - Thread-level parallelism (TLP)
 - Request-level parallelism (RLP)
- These require explicit restructuring of the application

计算机类别（截至2010年）

- **Personal** Mobile Device (PMD)
 - e.g. smart phones, tablet computers
 - Emphasis on energy efficiency and real-time
- **Desktop** Computing
 - Emphasis on price-performance
- **Servers**
 - Emphasis on availability, scalability, throughput
- **Clusters / Warehouse** Scale Computers
 - Used for “Software as a Service (SaaS)”
 - Emphasis on availability and price-performance
 - Sub-class: Supercomputers, emphasis: floating-point performance and fast internal networks
- **Embedded** Computers
 - Emphasis: price, power

计算机类别

| Parallelism

- Classes of **parallelism in applications**:
 - **Data-Level** Parallelism (DLP)
 - **Task-Level** Parallelism (TLP)
- Classes of **architectural parallelism**:
 - **Instruction-Level** Parallelism (ILP)
 - **Vector** architectures/**Graphic** Processor Units (GPUs)
 - **Thread-Level** Parallelism
 - **Request-Level** Parallelism

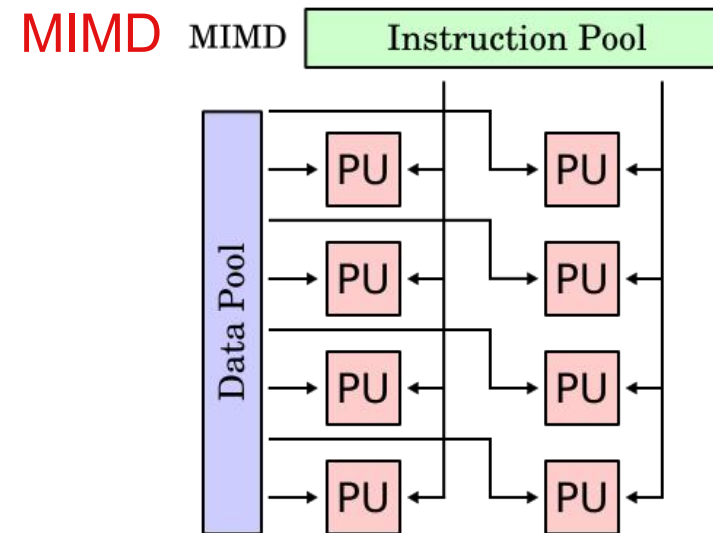
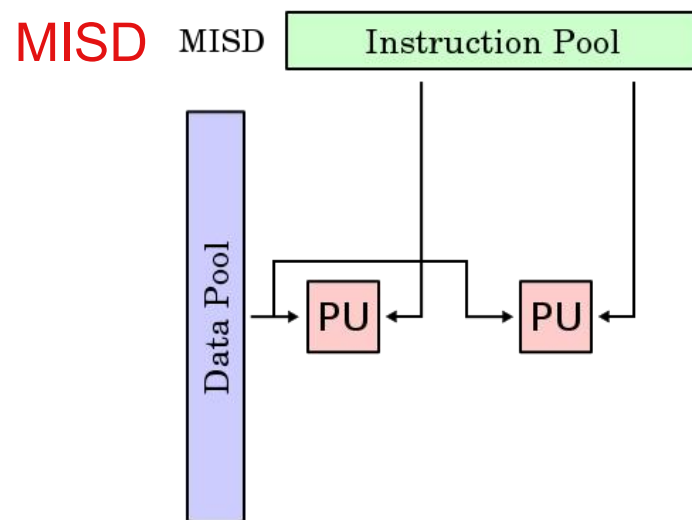
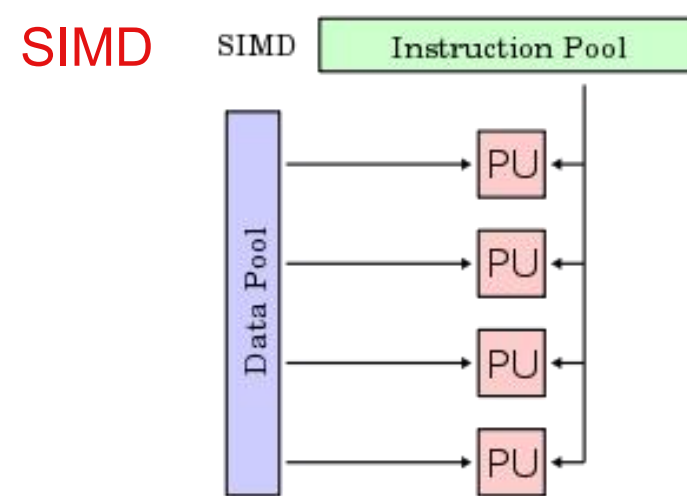
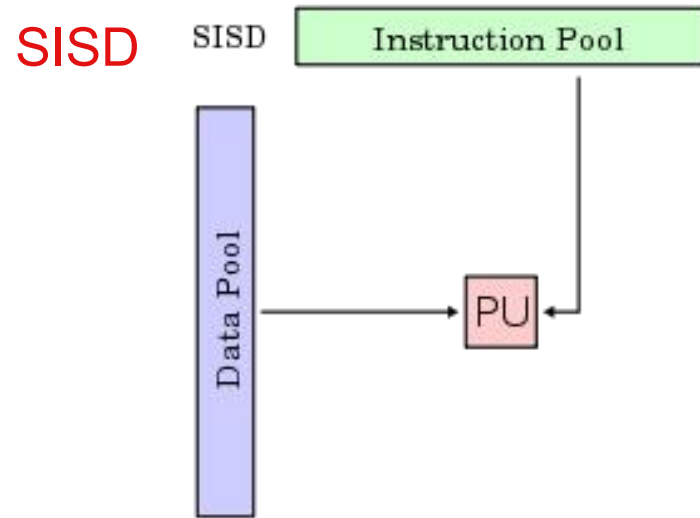
计算机类别

I Flynn's Taxonomy

- Single instruction stream, single data stream (SISD)
- Single instruction stream, multiple data streams (SIMD)
 - Vector architectures
 - Multimedia extensions
 - Graphics processor units
- Multiple instruction streams, single data stream (MISD)
 - No commercial implementation
- Multiple instruction streams, multiple data streams (MIMD)
 - Tightly-coupled MIMD
 - Loosely-coupled MIMD

计算机类别

| Flynn's Taxonomy



迫切需要更好的计算机体系结构!

- We need better computer architectures to fill the gaps
 - **Architecture**
 - von Neumann Architecture & Harvard Architecture
 - VLIW, SuperScalar
 - ?
 - **Instruction Set**
 - CISC, RISC
 - ?
 - **CPU & Memory**
 - Hierarchy of Memory: Cache, Memory, Disk, ...
 - ?
 - **Advanced Peripheral Devices**

过去的体系结构技术

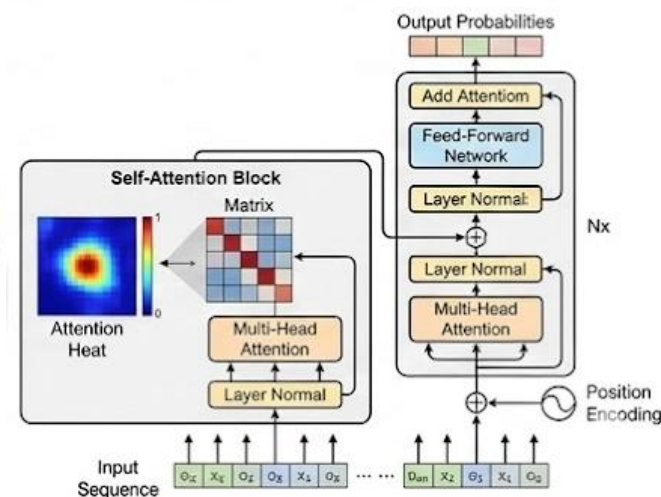
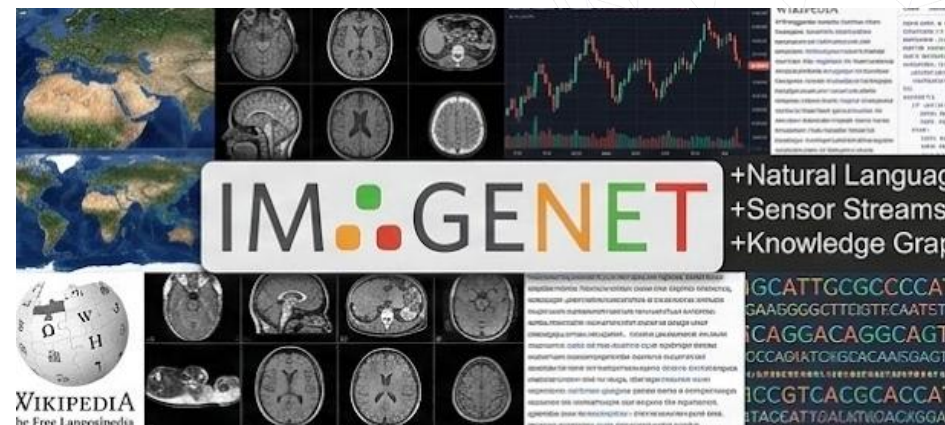
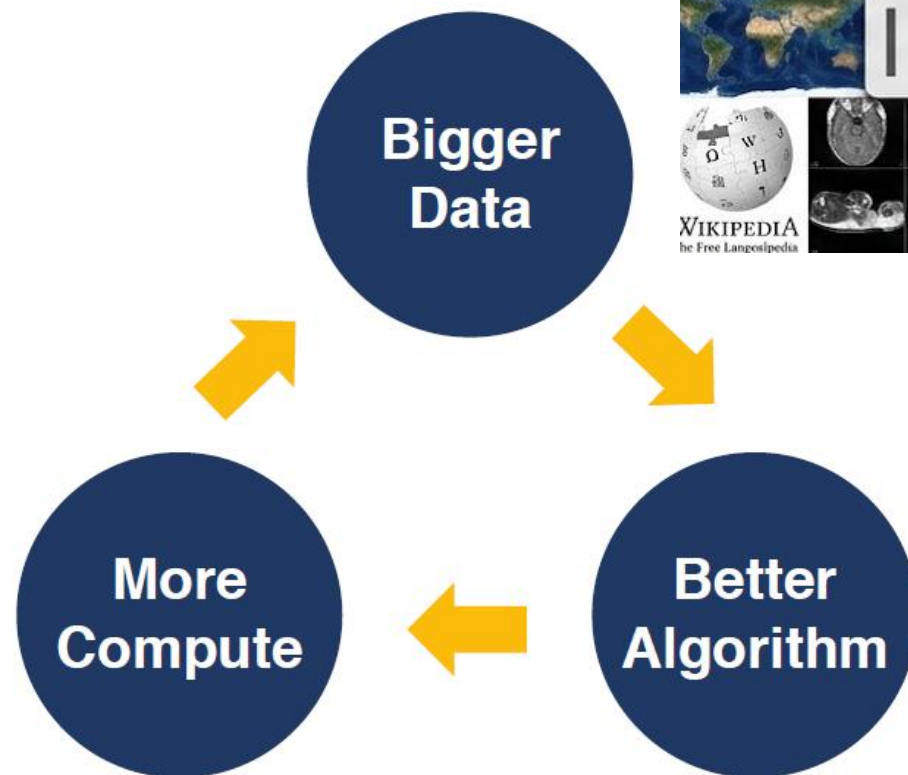
- **Pipeline**
 - Separate the execution of an instruction into several stages using separate resources
 - Overlap the execution of multiple instructions
 - e.g., we can have 5 stages: IF → ID → EX → MEM → WB
- **Branch Prediction**
- **Data Flow Analysis**: optimal instruction schedule
- **Speculative Execution**
- **Performance Balance**: Adjust organization and architecture to compensate for the mismatch among various components.

解决处理器速度与动态随机存取存储器速度之间的不匹配

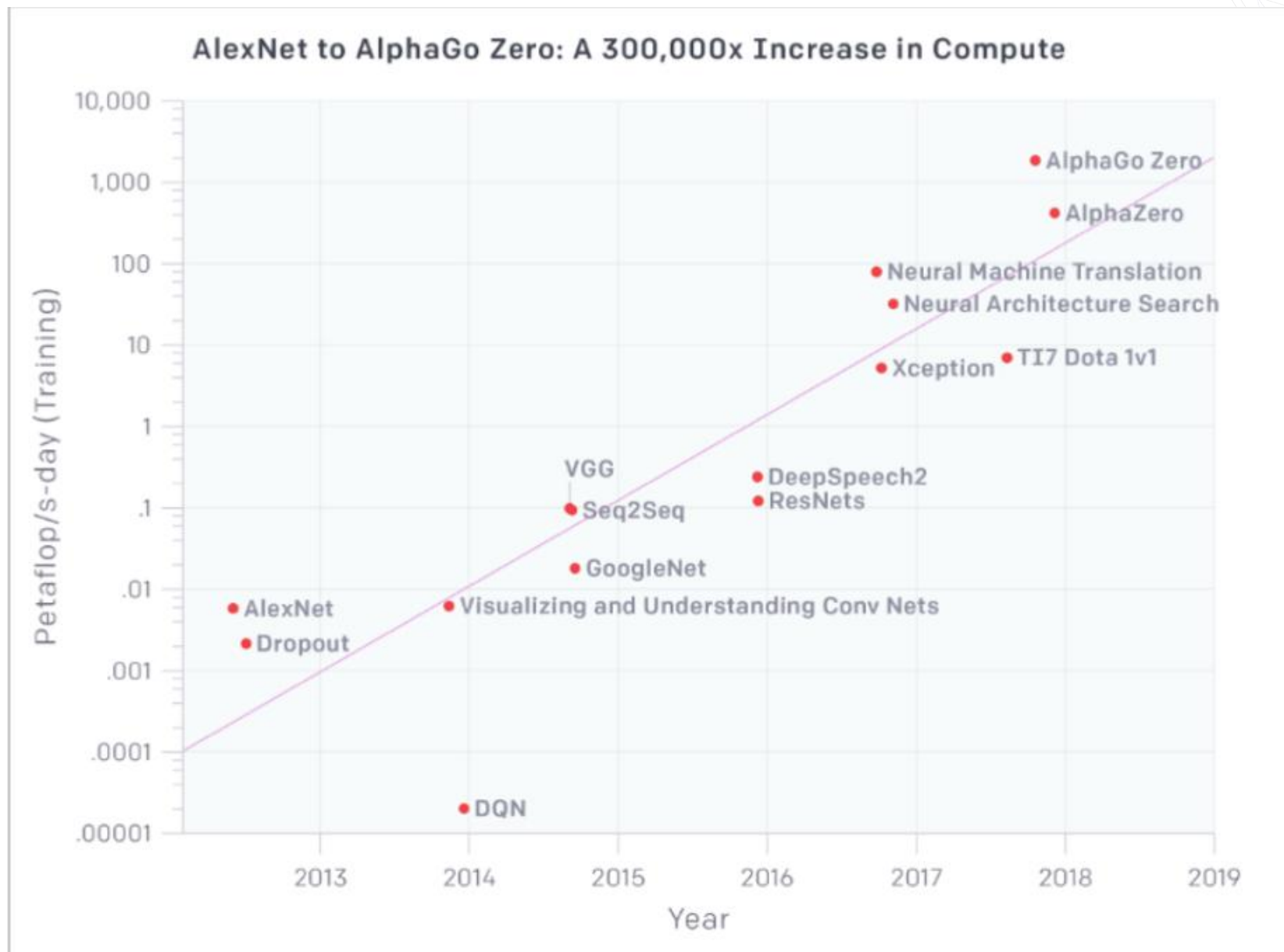
- Use **wide data buses** so we can retrieve more bits at the same time.
- Include a **cache /other buffer scheme** to make memory chip work more efficiently.
- Put **cache into processors**
- Use **high-speed buses** to interconnect processor and memory.
- **Near Data Computing, Processing in Memory.....**

AI Chip

Artificial Intelligence



为什么会有AI芯片 (Why AI Chip)



为什么会有AI芯片 (Why AI Chip)

Increasing Demand for Computing



Augmented Reality



Mobile Gaming



4K Video

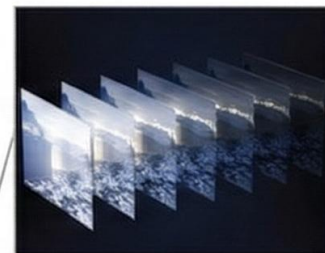
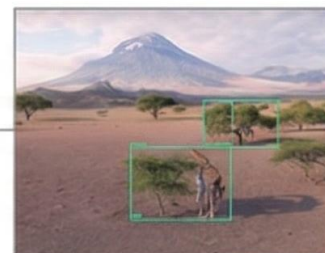


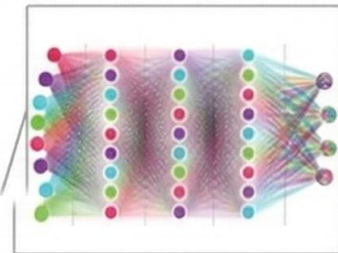
Image Processing



Object Tracking



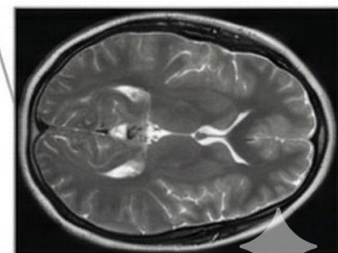
Obstacle Avoidance



Machine Learning



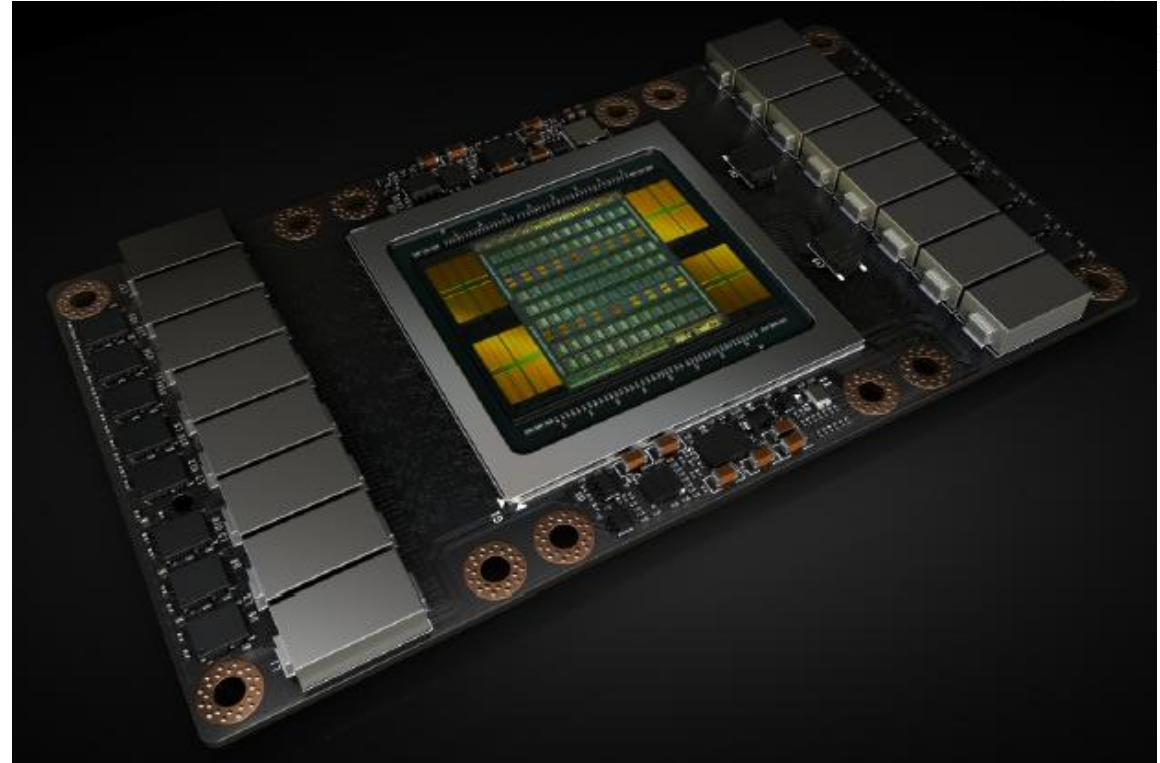
Genomics



Medical Imaging

NVIDIA GPU

- Volta V100 GPU
 - 21 billion transistors
 - Die size 815 mm²
 - TSMC 12 nm FinFET
 - 15.7 TFLOP/s of single precision (FP32) performance
 - 125 Tensor TFLOP/s of mixed-precision matrix-multiply-and-accumulate
 - TDP 300W

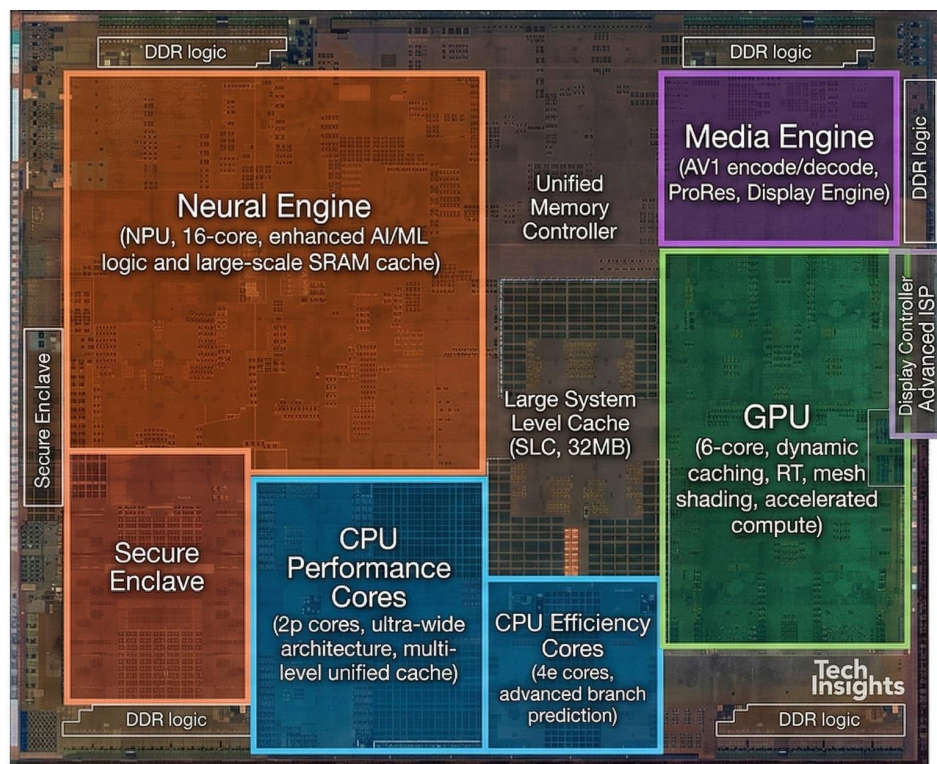


领域专用加速器 (Domain-Specific Accelerators)

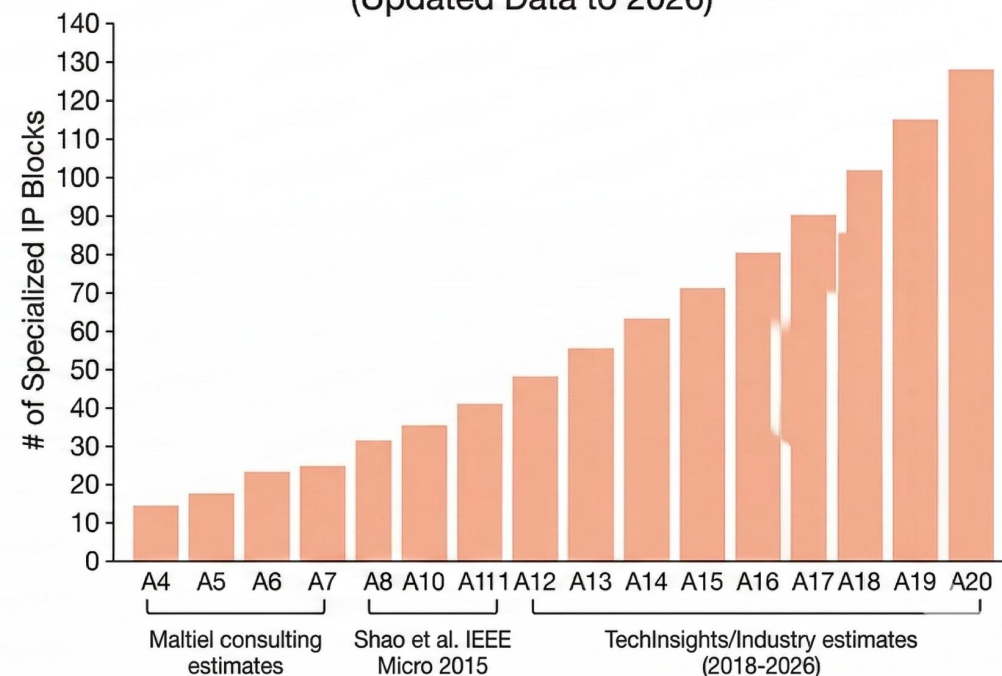
Customized hardware designed for a domain of applications



iPhone 17 Pro Max



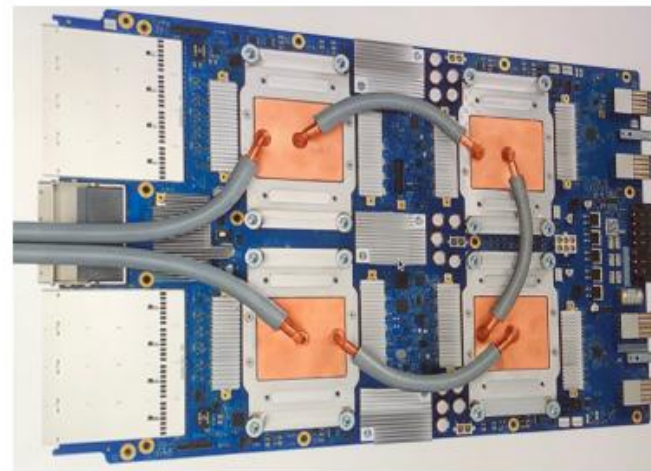
Growth of Specialized IP Blocks over Time
(Updated Data to 2026)



领域专用加速器 (Domain-Specific Accelerators)

Google TPU

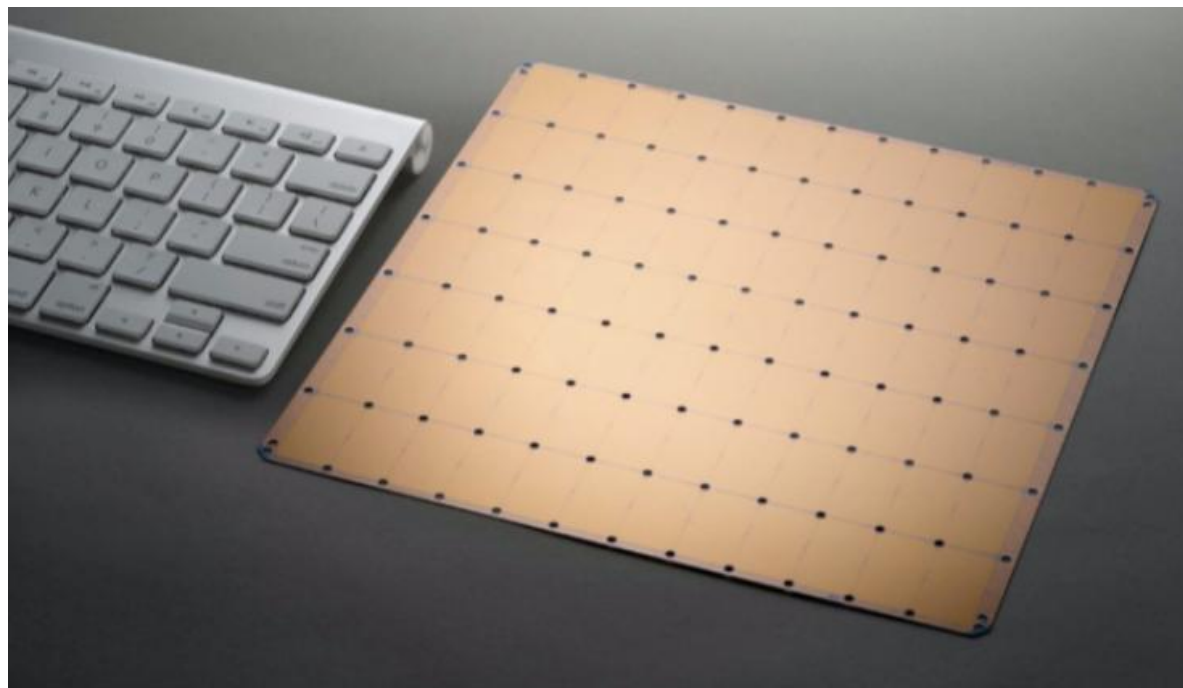
- Systolic-array-based architecture
 - V1: Inference only
 - V2: Training with bfloat
 - V3: 2x powerful than v2
- Edge TPU
 - Coral Dev Board
 - 4 TOPS
 - 2 TOPS/Watt
 - Supports TensorFlow Lite



领域专用加速器 (Domain-Specific Accelerators)

Cerebras: Wafer-Scale Deep Learning

- Largest Chip Ever Built!
- 46,225 mm² silicon
- 1.2 trillion transistors
- 400,000 optimized AI cores
- 18 GB of on-chip memory
- TSMC 16nm process



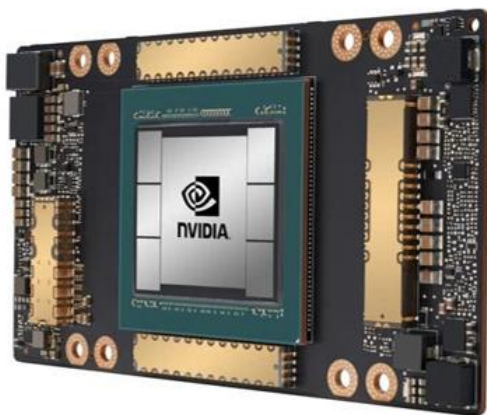
行业正在蓬勃发展



四类硬件平台



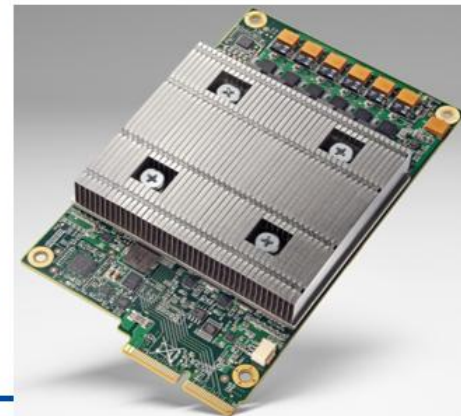
CPU



GPU



FPGA



ASIC



五种加速器芯片



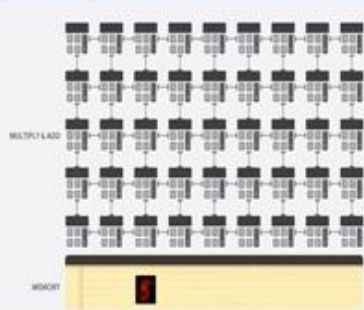
Instruction-based

High Generality

寒武纪

阿里含光

华为昇腾



Dataflow

High Utilization

Usually Combined

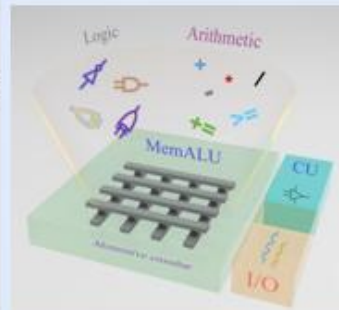
鲲云CAISA

Sambanova



Reconfigurable

Balance Flexibility and Efficiency



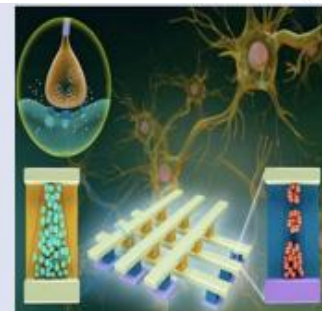
In-Memory Computing

High Bandwidth

IBM相变存内计算

台积电RRAM

英特尔NOR Flash



Brain-Inspired

AI Chip for Next Generation

清华大学天机

IBM TrueNorth

英特尔 Loihi

总结

- Technique advance made **dramatic performance improvement** in the past 40 years
 - **Processor**
 - logic capacity: increases about **30%** per year
 - performance: **2x** every **1.5** years
 - **Memory**
 - DRAM capacity: **4x** every **3** years, now **2x** every **2** years
 - memory speed: **1.5x** every **10** years
 - cost per bit: decreases about **25%** per year
 - **Disk**
 - capacity: increases about **60%** per year

Moore's Law will continue work in the near future?

- **Better architectures are needed for better usage of IC capacity**